

KOGI

Independent Worker Operating System

Master Distributed Spreadsheet System

KPMS · KPSS · KPRG · KPVW · KPCM · KPCL · KPEX · KLNK · KPID

Portfolio Spreadsheet · Link Network · Forest & Tree Architecture

Multi-Account · Multi-Identity · Multi-Profile · Cross-Portfolio Connections

Attribute	Value
Document Version	v1.0 — March 2026
Module Codes	KPMS · KPSS · KPRG · KPVW · KPCM · KPCL · KPEX · KLNK · KPID
Primary Languages	Go (services) · Rust (core) · Scala (kogi-engine) · TypeScript (UI)
Status	System Design — Active Development
Classification	Confidential — Kogi Platform Internal
Companion Documents	Portfolio System SDD v2.1 · kogi-bank SDD · kogi-community SDD v2.0 · kogi-game SDD · kogi-marketplace SDD · kogi-rms SDD · kogi-wms SDD

Executive Summary

The Kogi Master Distributed Spreadsheet System (KMDSS) is the central, sovereign data substrate of the entire Kogi Independent Worker Operating System. It is the unified, configurable, AI-augmented, distributed spreadsheet that every user, organization, collective, cooperative, and federation on the platform uses to record, manage, analyze, and connect every entity in their working, creative, financial, and collaborative life.

At its core, the KMDSS is architecturally simple: every entity in the Kogi ecosystem — programs, projects, resources, assets, artifacts, gigs, benefits, finances, relationships, governance actions, analytics events — is a row in one universal spreadsheet. Every property, metric, relationship, and computed value on that entity is a column. Every curated, filtered, grouped, or aggregated perspective on that data is a sheet. Every user owns their own root spreadsheet — their Master Portfolio Spreadsheet — and the entire Kogi network is the graph of interconnected spreadsheets, linked across users, identities, organizations, and federation nodes.

The KMDSS extends far beyond a conventional spreadsheet tool. It is:

- A living, event-sourced, CRDT-distributed data structure with append-only audit trails and nanosecond-precision VectorClock versioning
- A universal component registry where every Kogi entity is a typed, governed, policy-enforced row with full lifecycle management
- A multi-dimensional view engine that renders the same underlying data as 30+ specialized sheets, plus Kanban boards, Gantt charts, network graphs, treemaps, and pivot tables
- A computation engine with 20+ built-in analytical models producing health scores, risk scores, income projections, and AI-derived signals inline as computed columns
- A federated link network — the KLNK system — that maps the full graph of inter-user, inter-organization, and inter-portfolio connections across the entire platform, rendered as forests and trees of linked spreadsheet nodes
- A multi-account, multi-identity, multi-profile system allowing users to split their root spreadsheet across identities, merge contributions across profiles, and control exactly what is visible to whom
- A collaborative editing platform with real-time concurrent multi-user editing, CRDT-based conflict resolution, and contribution attribution for shared and crowdresourced portfolios
- An intelligence layer where the full kogi-engine stack — 20+ sub-engines — contributes computed columns, anomaly flags, match scores, risk signals, and Oba AI recommendations directly into every sheet

This document provides the complete, exhaustive system design for the KMDSS: every column, every sheet, the full link network architecture, the multi-identity model, all engine integrations, the API surface, persistence strategy, security model, and all open items and roadmap milestones.

Table of Contents

1. System Overview & Vision

1.1 The Master Spreadsheet Metaphor

Design Principle: Every entity in the Kogi ecosystem — programs, projects, resources, assets, artifacts, gigs, benefits, finances, relationships, governance actions, analytics events — is a row in one universal, scalable, configurable spreadsheet. The Portfolio Management System is that spreadsheet: the master, living, computed record of an independent worker's entire operational and creative world.

Every independent worker, autonomous organization, collective, cooperative, and federation on the Kogi platform owns a Master Portfolio Spreadsheet — a root-level spreadsheet workbook that is the authoritative, living record of everything they own, operate, create, manage, finance, and share. This is not a metaphor or a display abstraction. The spreadsheet IS the data model.

The Master Portfolio Spreadsheet is the application of the spreadsheet paradigm — one of the most universal and powerful information organization systems humans have invented — to the full domain of independent work. Where a conventional spreadsheet is a static grid of user-entered values, the Kogi Master Spreadsheet is:

- Event-sourced: every cell mutation is an immutable EventLog entry, never deleted
- Computed: hundreds of columns are derived, not entered — computed from other columns, child rows, external data, or AI engine signals
- Distributed: every spreadsheet is replicated across nodes via CRDT, enabling offline editing, multi-device access, and federated multi-user collaboration without conflicts
- Connected: spreadsheets are linked to each other via the KLNK Link Network, forming a graph of inter-portfolio connections that models the entire Kogi economy
- Policy-governed: every row, column, and cell respects a PermissionTier hierarchy, row-level security, column-level visibility, and a pluggable PolicyEngine

Spreadsheet Concept	Portfolio System Mapping
Workbook	PortfolioSystem — the root container holding all sheets for a user, org, or federation
Sheet	A named, scoped, filterable materialized view of PortfolioComponents (e.g. Projects Sheet, Finances Sheet, Benefits Sheet)
Row	A PortfolioComponent — any item, container, resource, artifact, asset, or derived entity
Column	A typed PortfolioColumn — a field, computed metric, relationship reference, or engine-derived signal on a component
Cell	The intersection of a PortfolioRow and a PortfolioColumn — a single typed, versioned, audited value

Spreadsheet Concept	Portfolio System Mapping
Formula	A ComputedColumn — a derived field calculated from other columns via the Computation Engine (e.g. Budget Remaining = Budget Allocated - Budget Spent)
Filter	A ViewFilter — a boolean predicate applied to rows (e.g. Status = Active, Type = Project, Owner = me)
Sort	A ViewSort — ordering applied to rows (e.g. sort by Health Score descending)
Group	A ViewGroup — hierarchical aggregation of rows by a shared column value
Pivot Table	A PivotView — cross-dimensional aggregation (e.g. Budget Spent by Owner x Program x Quarter)
Chart	A ChartView — visual rendering of a column's values (bar, line, gauge, treemap, network graph)
Freeze Panes	PinnedColumns — identity and key status columns always visible during horizontal scroll
Conditional Formatting	RowHighlightRule — color-coding rows based on column values (e.g. red for overdue, green for on-track)
Import / Export	KPEX — CSV, XLSX, JSON-LD export of any sheet; webhook push; kogi-dev API pull
Workbook Link	KLNK GraphEdge — a typed, directional link between two PortfolioRow stores across users or nodes

1.2 Module Codes & Services

Module	Code	Description	Language
Portfolio Management System	KPMS	Master orchestration layer: spreadsheet substrate, row/column model, sheet registry, view engine	Go + Rust
Portfolio Spreadsheet Substrate	KPSS	Low-level data engine: PortfolioRow, ColumnSchema, SheetDefinition, indexed store	Rust
Portfolio Component Registry	KPRG	Canonical registry of all PortfolioComponents system-wide; the master index	Go + PostgreSQL
View Engine	KPVW	Filter · Sort · Group · Pivot · Slice across any column dimension	Go
Computation Engine	KPCM	20+ analytical models; derived column computation; rollup and aggregation	Rust + Scala
Collaborative Editing	KPCL	CRDT-backed concurrent multi-user spreadsheet editing	Rust

Module	Code	Description	Language
Export & Integration	KPEX	CSV · XLSX · JSON · API · Embed · Webhook export of any sheet or view	Go
Link Network	KLNK	Inter-portfolio link graph: forests, trees, cross-user connections, visibility protocol	Go + Rust (GraphEngine)
Identity & Profile System	KPID	Multi-account, multi-identity, multi-profile root spreadsheet management and splitting	Go

1.3 Design Goals

Goal	Description
Universal Abstraction	Every platform entity — from a single task to a federation portfolio — is representable as a row in this system
Infinitely Configurable	Columns are addable, reorderable, hideable, pinnable, and computable. Sheets are addable, filterable, and shareable. Views are user-defined, template-driven, and AI-suggested
Event-Sourced & Auditable	Every cell mutation is appended to the component's EventLog. No data is ever deleted — only archived or superseded
Distributed-First	VectorClock + CRDT log (LWW + OR-Set) ensures that concurrent edits across nodes, collaborators, and federated portfolios converge correctly
AI-Augmented	Computed columns, smart defaults, anomaly flags, health scores, risk scores, match scores, and proactive suggestions generated by kogi-engine and surfaced directly in the spreadsheet
Policy-Governed	Every row, column, and cell respects a tiered PermissionTier system. Row-level and column-level access control allows fine-grained sharing
Composable & Extensible	The PortfolioPlugin trait and ToolBox integration allow developers and power users to extend the system with custom column types, computed models, and automated row generators
Collaborative	Multi-user real-time collaborative editing via CRDT; contribution attribution via ContributionRecord; shared portfolio governance via GovernanceProposal
Link-Networked	Every spreadsheet is a node in a global link graph. Users can connect their portfolios to others', view linked rows, subscribe to changes, and participate in federated economies
Multi-Identity	A single root spreadsheet can span multiple accounts, profiles, and identities, with configurable visibility, data partitioning, and cross-identity merge capabilities

2. System Architecture

2.1 Five-Layer Architecture

The Master Distributed Spreadsheet System is organized in five distinct layers, from the raw data substrate up to the user-facing view and intelligence layers. Each layer builds cleanly on the layer below, enabling both monolithic single-user deployments and massive federated multi-organization deployments to use the same codebase.

Layer	Name	Components & Responsibility
L0 — Substrate	Raw Component Store	PortfolioComponent (Rust core) · ComponentMetadata · ComponentData · EventLog · CrdtLog · VectorClock · GraphEdge · PolicyEngine · ToolBoxId refs
L1 — Registry	Component Index	KPRG Master Component Registry · ComponentIndex · OwnerIndex · TagIndex · TypeIndex · StatusIndex · FullTextIndex (SearchEngine) · LinkNetworkIndex
L2 — Spreadsheet Engine	KPSS Row/Column/Cell	PortfolioRow · ColumnSchema · SheetDefinition · CellStore · ColumnComputer · AggregationEngine · ViewFilter/Sort/Group
L3 — View Engine	KPVW View Layer	ViewFilter · ViewSort · ViewGroup · PivotView · ChartView · RowHighlightRule · PinnedColumns · BoardMode · ViewTemplate · SavedViews
L4 — Intelligence Layer	kogi-engine Signals	ComputedColumns · HealthScoreColumn · RiskScoreColumn · MatchScoreColumn · RecommendationColumn · AnomalyFlagColumn · Oba AI overlay
L5 — Application Layer	User-facing Apps	kogi-portfolio UI · kogi-office Boards · kogi-bank Finance Views · kogi-marketplace Asset Views · kogi-community Feed Integration · KPEX Export
L6 — Link Network	KLNK Graph	InterPortfolioLink · LinkForest · LinkTree · VisibilityProtocol · SubscriptionEdge · FederationEdge · ShadowRow · MirrorColumn
L7 — Identity Layer	KPID Multi-Identity	RootSpreadsheet · AccountMapping · ProfilePartition · IdentityAnchor · CrossIdentityMerge · VisibilityMask · SplitPolicy

2.2 Core Structural Primitives

Before describing the full row and column schemas, the following core structural primitives underpin every concept in this document:

Primitive	Type	Definition
PortfolioComponent	Rust struct	The universal node. Every row in any sheet is a PortfolioComponent. It is either an Item (leaf work

Primitive	Type	Definition
		entity) or a Container (organizing structure). Carries ComponentMetadata + ComponentData.
ComponentId	UUID	Globally unique, immutable, 128-bit identifier for every PortfolioComponent. Never reused, never modified.
PortfolioRow	Rust struct + Go view layer	A PortfolioComponent projected as a spreadsheet row: flat key-value map of ColumnId → TypedCellValue, computed columns resolved, relationships joined.
PortfolioColumn	Rust enum	A typed field definition. Variants: TextField NumberField CurrencyField DateField EnumField RelationField UserField TagField ComputedColumn AIColumn FormulaColumn AuditColumn
SheetDefinition	Go struct	A named, scoped, filterable view: base row type filter + visible column schema + default sort + default group + computed column set.
ViewDefinition	Go struct	A user-saved customization of a SheetDefinition: filters, sorts, groups, column widths, pinned columns, highlight rules, board config.
CellValue	Rust enum	The typed value at a row-column intersection. Variants: Text Number Currency Date Enum Relation User Tag Bool Json Null
EventLog	Rust VecDeque + PostgreSQL	Append-only, immutable log of every mutation on a PortfolioComponent. Every cell write is an EventLogEntry.
CrdtLog	Rust Vec	LWW + OR-Set CRDT operation log. Applied on merge to resolve concurrent edits without data loss.
VectorClock	Rust HashMap<NodeId, u64>	Logical clock per node. Provides causal ordering of all mutations across distributed nodes.
GraphEdge	Rust struct	A typed, directional relationship between two PortfolioComponents. Types: Hierarchy Dependency Link Contains Federation ResourceShare Attribution Investment Derives InterPortfolioLink
InterPortfolioLink	GraphEdge subtype	A link crossing the boundary between two different users' root spreadsheets. The atomic unit of the KLNK link network.

2.3 Component Taxonomy — Full Row Type Registry

The following is the complete taxonomy of PortfolioComponents — every possible row type that can appear in the master spreadsheet system. All types share the universal PortfolioRow schema (§4) but activate different column groups and carry type-specific payload structs.

Category	Row Type	Description	Key Columns Active
item:portfolio	Portfolio	A named top-level portfolio — the worker's entire work domain, or a specific practice area	mission, focus_areas, kpis, health_score, child_count, budget_rollup
item:portfolio	SubPortfolio	A scoped sub-portfolio nested within a parent Portfolio	parent_portfolio_id, scope, strategic_goals, program_rollup
item:program	Program	A strategic initiative grouping multiple related Projects toward a shared objective	objective, project_count, budget, kpi_achievement, program_alignment_score
item:project	Project	A bounded, deliverable-producing unit of work with methodology and sprints	project_type, methodology, sprint_velocity, backlog_size, completion_pct
item:resource	Resource	A human, financial, equipment, or service resource allocatable to work	resource_type, skills, availability, hourly_rate, utilization_pct
item:artifact	Artifact	A typed output document, file, design, code, or generated asset	artifact_type, file_refs, produced_by, maturity_score, version_count
item:asset	Asset	A capital or intellectual asset owned or managed	asset_type, valuation, currency, acquired_at, current_value, roi_pct
item:benefit	BenefitAccount	A portable benefit account (HSA, retirement, PTO, professional development)	benefit_type, provider, balance, ytd_contributions, coverage_gap_flag
item:gig	Gig	A single gig engagement generating income	platform, earnings_gross, tips, portable_contribution, mileage_deduction
item:contract	Contract	A formal agreement with a client, cooperative, or employer	client_ref, status, payment_terms, deliverables, escrow_status
item:job	Job	A longer-term employment or consulting engagement	rate, hours, organization_ref, benefits_linkage, contract_type
item:task	Task	A single atomic unit of work within a project backlog	story_type, priority, story_points, assigned_to, sprint_ref, dependencies

Category	Row Type	Description	Key Columns Active
item:campaign	Campaign	An equity crowdfunding, grants, or microfinancing campaign	campaign_type, target_amount, raised_amount, investor_count, close_date
item:grant	Grant	A grant application or awarded grant with milestones and reporting	source, status, awarded_amount, disbursed_amount, milestone_refs
item:investment	Investment	An equity stake, revenue-share instrument, or pool investment	instrument_type, amount, units, vesting_schedule, return_to_date
item:profile	Profile	A user or organization profile — surfaced in community and directory sheets	display_name, skills, reputation_score, connection_count, link_count
container:binder	Binder	A logical collection of items organized by shared purpose	item_count, coverage_pct, last_activity
container:book	Book	A structured, navigable document-like container with typed subtypes	book_type, section_count, consistency_score, last_updated
container:record	Record	A formal, auditable record container	record_type, integrity_score, compliance_flags
container:folder	Folder	A general-purpose hierarchical container	item_count, depth, orphan_count, organisation_score
container:registry	Registry	A structured, searchable catalog of items	entry_count, category, last_indexed
container:archive	Archive	Deep-storage container for completed or historical items	archived_item_count, restore_available, retention_policy

3. Multi-Account, Multi-Identity & Multi-Profile Architecture (KPID)

3.1 Overview

One of the most powerful and distinctive features of the Kogi Master Distributed Spreadsheet System is that a single worker's root spreadsheet can span multiple accounts, profiles, and identities — while the KLNK Link Network shows how all of these identities interconnect with each other and with the portfolios of other users. This is the KPID (Portfolio Identity) system.

Design Principle: Identity on Kogi is not one-to-one. A person may be a freelance developer under one identity, an artist collective member under another, a cooperative officer under a third, and an anonymous public voice under a fourth. The Master Spreadsheet respects this reality: a root spreadsheet is not tied to one account — it is tied to one sovereign entity (a person, org, or collective), which may be expressed through multiple accounts and profiles.

3.2 Identity Hierarchy

Concept	Description	Relationship to Spreadsheet
Sovereign Entity	The real-world person, organization, collective, or cooperative that owns the root spreadsheet. Has exactly one root PortfolioSystem instance as their master store.	One-to-one: one Sovereign Entity = one Master Portfolio Spreadsheet root
Account	A login credential set (email+password, OAuth, passkey) associated with a Sovereign Entity. A Sovereign Entity may have multiple accounts (e.g. personal email + work email + social OAuth).	Accounts are authentication surfaces. All authenticated accounts for the same Sovereign Entity share read/write access to the full root spreadsheet, subject to per-account VisibilityMask.
Profile	A curated, public-facing identity projection of the Sovereign Entity. Profiles select which portfolio components are visible, which skills are highlighted, and which links are shown.	A Profile is a ViewDefinition applied to the Public sheet. Each profile has its own VisibilityMask defining which rows are public, followers-only, or private.
Identity	A named, purpose-specific persona with a separate @handle, display name, avatar, and optional pseudonymity. E.g. @jordan-dev (professional), @jordan-art (creative), @studiocollective (org).	Each Identity maps to a ProfilePartition — a subset of rows tagged to that identity. Rows can belong to multiple identities simultaneously.
ProfilePartition	A logical partition of the root spreadsheet's rows, owned by one Identity. Rows in a partition carry an identity_tag[] that maps them to one or more profiles.	ProfilePartition defines which rows are visible on which profile. Cross-partition visibility is governed by VisibilityMask rules.

Concept	Description	Relationship to Spreadsheet
VisibilityMask	A per-account or per-profile filter specifying which rows and columns are visible to which observer.	Applied at query time: every GET request against the spreadsheet is filtered through the caller's VisibilityMask before returning data.
SplitPolicy	A governance policy governing how data is partitioned across identities. Enforces that certain rows (e.g. financial data, sensitive contracts) are only visible to specific identity tiers.	SplitPolicy is enforced by the PolicyEngine at the Substrate layer — no query can bypass it.

3.3 Root Spreadsheet Splitting

When a Sovereign Entity creates multiple identities or operates across multiple organizational roles, their root spreadsheet can be "split" — logically partitioned across identity boundaries — while remaining a single unified data store underneath. Splitting is a view-layer operation: the data is never physically duplicated.

3.3.1 Split Operations

Operation	Description	Reversible?
CreateIdentity	Create a new Identity (@handle) under the Sovereign Entity. Provisions a new ProfilePartition. No rows are moved — they are tagged.	Yes — identities can be merged or deleted
TagRowToIdentity	Assign one or more identity_tags to a row, making it part of one or more ProfilePartitions. A row may belong to multiple partitions simultaneously.	Yes — tags can be added/removed
SetVisibilityMask	Define which rows/columns are visible to observers of a specific identity profile. Observer types: Public Followers Connections Collaborators Self	Yes — masks can be updated
IdentityIsolation	Mark an identity as Isolated: rows tagged only to this identity are completely hidden from all other identity views, even when authenticated as the same Sovereign Entity.	Yes — isolation can be lifted
CrossIdentityMerge	When the same Sovereign Entity has operated multiple identities and decides to unify them, CrossIdentityMerge re-tags all rows from the source identity to the target, preserving full history.	Partial — history preserved; cannot un-merge
IdentityHandoff	Transfer one Identity (including all its tagged rows) to a different Sovereign Entity — e.g.	No — creates a new root spreadsheet for recipient

Operation	Description	Reversible?
	when a solo practitioner spins a collective identity into an independent organization.	
FederatedIdentity	Link an identity to an external platform identity (GitHub, LinkedIn, DID/verifiable credential). Federated identities can receive link connections from external platforms.	Yes — links can be revoked

3.3.2 ProfilePartition Schema

Each ProfilePartition is a lightweight struct referencing the root spreadsheet. It carries no data of its own — it is a view configuration pointing into the master component registry.

Field	Type	Description
partition_id	UUID	Unique identifier for this partition
identity_handle	String	@handle for this identity (unique within Sovereign Entity)
sovereign_entity_id	EntityId	Owning Sovereign Entity
display_name	String	Public display name for this identity
avatar_ref	Option<FileId>	Profile image for this identity
bio	Option<String>	Bio text for public profile
visibility_mask	VisibilityMask	Row and column visibility rules for public observers
identity_tag	String	Tag applied to rows belonging to this identity
isolation_level	IsolationLevel	None SoftIsolated HardIsolated — controls cross-identity data access
linked_accounts	Vec<AccountId>	Accounts that can authenticate as this identity
public_sheet_view_id	ViewId	The ViewDefinition rendered when someone visits this identity's public profile
follower_sheet_view_id	ViewId	View rendered for followers/subscribers
connection_sheet_view_id	ViewId	View rendered for trusted connections
link_network_node_id	LinkNodeId	This identity's node in the KLNK link network graph
created_at / updated_at	DateTime	Timestamps

3.4 Multi-Account Authentication & Spreadsheet Access

A Sovereign Entity may have any number of accounts. Each account has a configurable access scope defining which partitions it can read and write. This allows fine-grained separation — e.g. a

"client-facing" account that can only see the Work & Gigs partition, while the full account has access to finances and benefits.

Account Type	Default Access Scope	Use Case
Primary Account	Full root spreadsheet — all partitions, all columns	Personal daily use; full portfolio management
Work Account	Work & Gigs + Projects + Contracts partitions; financial summary only	Share with collaborators or embed in a work context
Organization Account	Org-tagged rows + shared portfolio components	When acting as officer or member of an org with a separate login
Public Account	Only rows where visibility = Public in all partitions	Published portfolio / showcase presence
API Account	Scoped by API key permissions — read-only or specific sheet access	Third-party integrations, BI tools, automated pipelines
Delegated Account	Defined by AccountDelegation policy — can act on behalf of Sovereign Entity within policy bounds	Virtual assistants, agents, authorized representatives

4. Universal Row Schema (PortfolioRow)

Every row in the master spreadsheet implements the PortfolioRow schema — a superset of ComponentMetadata + ComponentData + all derived and computed fields. Rows are typed: the active columns for a given row are determined by its ItemType or ContainerType, but the underlying storage schema is universal so that cross-type aggregation, filtering, and sorting works uniformly across all sheets.

4.1 Column Group Overview

Column Group	Column Count	Description
Identity	9	Immutable identifier, slug, type discriminants, display fields, color, icon
Taxonomy	8	Type, subtype, category, labels, tags, themes, domain, lifecycle_stage
Lifecycle	8	Status, state, visibility, lifecycle_stage, timestamps, soft-delete
Ownership & Users	12	Owners, editors, contributors, viewers, watchers, subscribers, followers, investors, donors
Relationships	11	Parent, children, dependencies, dependents, linked_ids, federation_edges, link_network_edges
Identity & Profile	8	Identity_tags, partition_ids, visibility_masks, public_view_id, account_visibility
Version Control	5	Version, version_history, last_actor, bump_type, changelog
Timeline & Schedule	12	Start, end, due, estimated_duration, actual_duration, progress_pct, schedule_variance, milestones
Financial & Budget	16	Budget_allocated, budget_spent, budget_remaining, utilization_pct, revenue, expenses, profit, roi, rate, rate_type, tax_category, invoices, bank_account_ref
Resource Allocation	8	Resource_type, capacity, allocated, consumed, utilization_pct, over_committed, allocation_refs
Work & Delivery	10	Story_type, priority, story_points, sprint_ref, velocity, throughput, cycle_time, lead_time, acceptance_criteria, delivery_evidence
Governance & Policy	9	Policy_ids, governance_model, approval_status, compliance_flags, risk_flags, charter_ref, regulatory_tags, audit_trail
Analytics & Metrics	18	Health_score, risk_score, match_score, views, clicks, ctr, shares, followers, kpi_refs, okr_refs, velocity, cycle_time, collaboration_score, sentiment_score, anomaly_flags

Column Group	Column Count	Description
Visibility & Access	5	Visibility, access_list, invite_ids, share_tokens, embed_config
Tags & Discovery	6	Tags, hashtags, topics, labels, categories, search_keywords
AI & Engine Signals	10	Oba_hint, anomaly_flags, recommendation_score, predicted_completion, budget_risk_flag, schedule_risk_flag, match_score, income_projection, grant_match_score, benefits_gap_flag
Collaboration	8	Contributor_count, contribution_types_open, review_policy, attribution_weights, merge_conflicts, governance_model, distribution_pending
Benefits	11	Benefit_type, provider, balance, ytd_contributions, contribution_sources, vesting_pct, pto_accrued, coverage_gap_flag, tax_treatment, tax_savings, claim_count
Gig & Contract	11	Platform, engagement_type, earnings_gross, tips, portable_contribution, mileage, mileage_deduction, payment_status, contract_status, deliverables, escrow_status
Link Network	9	Link_count, inbound_link_count, outbound_link_count, linked_portfolio_ids, link_visibility, shadow_row_count, mirror_column_count, federation_node_ids, link_depth

4.2 Identity Column Group

Column	Type	Description
component_id	UUID (immutable)	Globally unique immutable identifier for this row. Never changes after creation.
slug	String (unique/owner)	URL-safe short identifier. Unique within the owner's namespace. Used in API paths and embed URLs.
component_type	ComponentType enum	Item Container — top-level type discriminant controlling which column groups are active
item_type	Option<ItemType>	Portfolio Program Project Resource Artifact Asset BenefitAccount Gig Contract Job Task Campaign Grant Investment Profile
container_type	Option<ContainerType>	Binder Book Record Folder Registry Archive
name	String	Primary display name of the component. Shown on all cards, search results, and row headers.

Column	Type	Description
display_name	Option<String>	Override display name for compact views (e.g. short title for board cards, abbreviated name for columns)
icon	Option<String>	Icon reference: emoji character, icon set key (e.g. "heroicons/folder"), or custom URL
color	Option<HexColor>	Color tag for board and calendar views. Used in swimlane backgrounds, card accent bars, and calendar event color.

4.3 Lifecycle Column Group

Column	Type	Values / Description
status	ComponentStatus	Draft Active Paused Completed Archived Cancelled Suspended UnderReview Rejected Custom(String)
state	ComponentState	Initializing Configured Running Idle Blocked Failing Recovering Migrating Locked Sealed Custom(String)
visibility	Visibility	Private Protected Internal Public Unlisted DraftOnly Custom — governs link network discovery and public sheet visibility
lifecycle_stage	LifecycleStage	Concept Planning Execution Review Delivery Complete Post-Mortem — semantic stage overlay on top of status
created_at	DateTime<Utc>	Row creation timestamp. Immutable. Set by the Substrate layer on create_item() or create_container().
updated_at	DateTime<Utc>	Last mutation timestamp. Auto-updated on every cell write. Used as primary sort key for "recently updated" views.
archived_at	Option<DateTime<Utc>>	Archive timestamp. Set on transition to Archived status. Null for non-archived rows.
deleted_at	Option<DateTime<Utc>>	Soft-delete timestamp. Set on soft-delete. Rows with deleted_at are excluded from all views unless explicitly queried via SHT-028 Archive Sheet.

4.4 Financial & Budget Column Group (Extended)

The financial column group is the most comprehensive of all column groups, reflecting the centrality of financial management to the independent worker experience. These columns are active on all row types that carry financial data.

Column	Type	Applies To	Computation / Notes
budget_allocated	Option<Decimal>	Program, Project, Gig, Contract	Manually set or derived from ResourceAllocation. Null if no budget defined.
budget_spent	Decimal	Program, Project, Gig	Cumulative spend recorded via record_spend(). Auto-updated on each expense transaction.
budget_remaining	ComputedColumn	Program, Project	= budget_allocated - budget_spent. Negative if over budget.
budget_utilization_pct	ComputedColumn	Program, Project	= (budget_spent / budget_allocated) x 100. Flagged amber at 80%, red at 100%.
currency	CurrencyCode	All financial rows	ISO 4217 currency code. Multi-currency support: all values normalized to user's base currency for aggregation.
rate	Option<Decimal>	Gig, Contract, Job, Resource	Hourly, daily, project, or gig rate. Source for earnings computation.
rate_type	RateType	Gig, Contract, Job	Hourly Daily Weekly Monthly ProjectFixed PerUnit Revenue% Subscription
revenue	Decimal	Gig, Contract, Job, Campaign, Investment	Cumulative revenue generated. Updated by kogi-bank on payment receipt.
expenses	Decimal	Project, Program, Asset, Gig	Cumulative expenses. Updated by kogi-bank on expense transaction.
profit	ComputedColumn	All financial rows	= revenue - expenses. Key P&L metric.
roi	ComputedColumn	Asset, Investment, Campaign	= (revenue - budget_spent) / budget_spent x 100. Return on Investment percentage.
tax_category	TaxCategory	All financial rows	Business SelfEmployment PassThrough Exempt Investment Benefit — drives tax report generation
payment_terms	Option<String>	Contract, Gig, Job	Net-30, Net-60, milestone, upfront, subscription — surfaced in Invoices Sheet

Column	Type	Applies To	Computation / Notes
invoice_refs	Vec<InvoiceId>	Contract, Gig, Job	References to kogi-bank invoices linked to this component. Count shown as badge.
payment_status	PaymentStatus	Contract, Gig, Job	Unpaid Partial Paid Overdue Disputed. Synced from kogi-bank invoice status.
bank_account_ref	Option<AccountId>	All financial rows	kogi-bank account receiving or funding this component's transactions.

4.5 AI & Engine Signal Column Group

These columns are written exclusively by the kogi-engine sub-systems via the plugin writeback protocol. They are read-only from the user's perspective. They are cached in the CellStore with a TTL and invalidated on relevant EventLog events.

Column	Source Engine	Description
health_score	AnalyticsEngine + RiskEngine	PortfolioHealth model output (0-100): composite of delivery, financial, risk, resource, engagement, governance, and benefit dimensions. Primary AI column.
risk_score	RiskEngine	Probability-weighted aggregate of risk_flags (0-100). Lower is better. Drives Red/Amber/Green status indicator.
match_score	MatchEngine	Relevance score for this row in the context of a current search or opportunity query. Context-dependent.
recommendation_score	RecommendationEngine	How highly this row is recommended to the current viewer based on their persona and interaction history.
predicted_completion_date	OptimizationEngine	Monte Carlo simulation-derived completion date estimate with confidence interval. Project rows only.
schedule_risk_flag	RiskEngine	Boolean: true if predicted_completion_date exceeds due_date at >50% probability.
budget_risk_flag	RiskEngine	Boolean: true if projected budget_spent at current burn rate will exceed budget_allocated before project end.
income_projection	IncomeProjection model	Projected income from this row over the next 30/60/90 days. Gig, Contract, Investment rows only.

Column	Source Engine	Description
grant_match_score	GrantEngine	Eligibility-weighted match score for open grants in the registry. Grant rows only.
benefits_gap_flag	BenefitsEngine	Coverage gap detected for this benefit type. BenefitAccount rows only.
oba_hint	Oba AI overlay	Latest Oba AI annotation for this row — surfaced as inline tooltip or sidebar card. Plain text; updated on each Oba analysis cycle.
anomaly_flags	TelemetryEngine	Vec<AnomalyFlag> — engine-detected anomalies: budget spike, velocity drop, engagement cliff, risk escalation. Each flag carries type, severity, detected_at.

4.6 Link Network Column Group

These columns describe a row's position and connectivity in the KLNK Link Network. They are computed from the GraphEdge store and updated on every InterPortfolioLink create/delete event.

Column	Type	Description
link_count	ComputedColumn (u32)	Total outbound + inbound inter-portfolio links on this row. High link_count indicates a highly networked component.
inbound_link_count	ComputedColumn (u32)	Number of other users' spreadsheets that link to this row. Measure of external interest and reputation.
outbound_link_count	ComputedColumn (u32)	Number of rows in other users' spreadsheets that this row links to. Measure of connectivity.
linked_portfolio_ids	Vec<PortfolioId>	IDs of root spreadsheets that have an active InterPortfolioLink to/from this row.
link_visibility	LinkVisibility	Who can see this row's links: Public Followers Connections Private. Controls KLNK discovery.
shadow_row_count	ComputedColumn (u32)	Number of ShadowRows created in other users' spreadsheets that mirror this row.
mirror_column_count	ComputedColumn (u32)	Number of MirrorColumns from this row visible in linked spreadsheets.
federation_node_ids	Vec<NodeId>	CRDT federation nodes that replicate this row. Empty for non-federated rows.
link_depth	ComputedColumn (u32)	Shortest path distance in the link graph to the farthest connected portfolio. 0 = isolated, 1 = direct connection, 2+ = second-degree.

5. The Link Network, Forest & Tree Architecture (KLNK)

5.1 Overview

The KLNK Link Network is the global inter-portfolio graph of the Kogi platform. It is the distributed data structure that maps every connection between every portfolio, portfolio component, profile, and identity across every user, organization, collective, cooperative, and federation on the platform.

Design Principle: If the Master Portfolio Spreadsheet is the node, the KLNK Link Network is the edge set. Every time a user connects their portfolio to another's — by following, subscribing, collaborating, investing, linking a dependency, or joining a federation — they are adding a typed, directional edge to this global graph. The network of all these edges is the Kogi economy graph: the complete, queryable picture of how all independent work on the platform is interconnected.

The KLNK system models the link network at four levels of granularity:

- Component-level links: individual PortfolioComponents linked to individual components in other users' spreadsheets
- Portfolio-level links: one user's entire portfolio linked to another's — broad connections establishing general interest or collaboration
- Identity-level links: one identity (@handle) connected to another, establishing the social graph
- Federation-level links: entire PortfolioSystem instances (nodes) federated for CRDT sync and cross-node governance

5.2 KLNK Graph Model

Concept	Description	Data Model
LinkNode	Any entity that can be a vertex in the KLNK graph: a PortfolioComponent, Profile, Identity, Organization, Portfolio, or Federation Node.	LinkNode { node_id: NodeId, node_type: LinkNodeType, owner_entity_id: EntityId, visibility: LinkVisibility, metadata: ComponentMetadata }
LinkEdge	A typed, directional edge between two LinkNodes in the KLNK graph. The atomic unit of inter-portfolio connection.	LinkEdge { edge_id: EdgId, from_node: NodeId, to_node: NodeId, edge_type: LinkEdgeType, visibility: LinkVisibility, weight: f64, created_at: DateTime, metadata: Map<String, Value> }
LinkForest	The complete set of all LinkTrees rooted at a given Sovereign Entity. A user's "link forest" is all the trees that fan out from all their identities and portfolio components.	LinkForest { root_entity_id: EntityId, trees: Vec<LinkTree>, total_node_count: u64, total_edge_count: u64, depth_distribution: Map<u32, u64> }
LinkTree	A rooted, directed subgraph of the KLNK graph rooted at a single	LinkTree { root_node: NodeId, edges: Vec<LinkEdge>, nodes: Vec<LinkNode>,

Concept	Description	Data Model
	LinkNode. The tree represents the reachable graph from that node following all outbound edges up to a configured depth.	max_depth: u32, edge_type_filter: Option<Vec<LinkEdgeType>> }
ShadowRow	When User A links their portfolio component to User B's component, a ShadowRow is created in User B's spreadsheet — a read-only reflection of the linked component from User A's spreadsheet, rendered as a row in the relevant sheet.	ShadowRow { shadow_id: UUID, source_component_id: ComponentId, source_owner_entity_id: EntityId, visible_columns: Vec<ColumnId>, update_policy: ShadowUpdatePolicy, last_synced: DateTime }
MirrorColumn	A subset of columns from a ShadowRow made visible as a ComputedColumn in the host spreadsheet's view. User B can choose to mirror specific columns from User A's linked component into their own sheet.	MirrorColumn { mirror_id: UUID, shadow_row_id: UUID, source_column_id: ColumnId, target_column_id: ColumnId, transform: Option<ColumnTransform> }

5.3 Link Edge Types

Edge Type	Direction	Description	Creates ShadowRow?
Follows	A → B	User A follows User B's profile or portfolio component. A sees B's public updates in their feed. B's follower_count increases.	No — subscription only
Subscribes	A → B	A paid or privileged follow. A receives subscriber-tier content from B's portfolio. Governed by B's subscription terms.	Optional — if B grants subscriber ShadowRow access
Watches	A → B (component)	A watches a specific portfolio component of B. A receives all state-change notifications for that component.	Optional — watcher may create a ShadowRow
Collaborates	A ↔ B	A bidirectional collaboration link. A and B are co-contributors on a shared portfolio component. Both have ContributionRecord entries.	Yes — mutual ShadowRows for shared component
InvestedIn	A → B (campaign/asset)	A has committed capital to B's campaign, equity round, or revenue share instrument. A receives investor-tier data access.	Yes — investor ShadowRow with financial columns visible

Edge Type	Direction	Description	Creates ShadowRow?
Donated	A → B (campaign)	A made a donation to B's campaign or cause. Non-equity, non-return donation tracked in link graph.	No — ledger entry only
Contracted	A → B	A has an active or historical contract engagement with B. Both parties' contract rows are linked bidirectionally.	Yes — both parties see the deal in their Contracts sheet
DependsOn	A → B (component)	A's portfolio component has a declared dependency on B's component. Used for cross-portfolio critical path analysis.	Yes — B's component appears as dependency row in A's graph
ResourceShares	A → B (resource)	A shares a portfolio resource (template, playbook, tool, data, compute) with B at a specified access level.	Yes — ShadowRow of the shared resource in B's Resources sheet
FederationPeer	A ↔ B (system)	Two PortfolioSystem nodes federated for CRDT sync. All federated components replicate across the link.	Yes — all replicated components appear as rows in both systems
OrgMembership	User → Org	A user is a member of an organization. The org's shared portfolio components are visible to the member.	Yes — org's shared components appear in member's Shared Portfolios sheet
Mentors	A → B	A has a mentorship relationship with B. A's relevant portfolio components are visible to B as learning resources.	Optional — configured by mentor
Endorses	A → B (skill/compo nent)	A endorses a specific skill or portfolio component of B. Contributes to B's reputation_score.	No — endorsement record only
References	A → B (component)	A explicitly references B's component in their own portfolio documentation or proposal.	No — lightweight citation link
CampusLink	Platform → User	A platform-initiated link connecting a user to a platform resource, template, or grant. Used for onboarding flows.	Yes — platform resource appears as ShadowRow

5.4 Forest & Tree Rendering

The KLNK system provides multiple rendering modes for the link graph, each optimized for different use cases. All modes query the same underlying LinkEdge store; they differ only in how they traverse, filter, and visualize the graph.

Rendering Mode	Description	Max Depth	Primary Use Case
Ego Network View	A force-directed graph centered on the current user's root identity node, showing all first-degree and second-degree connections as a radial network.	Depth 2	Social graph overview; discover mutual connections; identify collaboration opportunities
Link Forest View	All LinkTrees rooted at the current user's identities and portfolio components, rendered side-by-side as a multi-root forest layout.	Depth 3	Full connectivity overview; identify isolated components; find bridge nodes
Component Tree View	A single LinkTree rooted at a specific portfolio component, showing all inbound and outbound links as a tree with branching paths.	Depth 5	Understand a single component's full network context; due diligence for deals
Dependency Tree View	A filtered LinkTree showing only DependsOn edges. Rendered as a top-down dependency diagram with critical path highlighting.	Unlimited (cycle-safe)	Cross-portfolio critical path analysis; dependency risk assessment
Investment Graph	A filtered graph showing InvestedIn and Contracted edges only. Reveals the financial network of deals, investments, and capital flows.	Depth 3	Investor portfolio visualization; deal network analysis
Federation Graph	A system-level graph showing FederationPeer edges between PortfolioSystem nodes. Rendered as a network topology diagram.	Full graph	Platform infrastructure overview; CRDT sync health monitoring
Organization Tree	An org chart-style tree showing OrgMembership edges from a root organization down through all member users and sub-organizations.	Depth 4	Org structure visualization; governance participation overview
Collaboration Web	A bipartite graph showing Collaborates edges between users and shared portfolio components.	Depth 3	Team composition analysis; contribution attribution visualization
Resource Sharing Map	A directed graph of ResourceShares edges showing how resources (templates, tools, data) flow across users and organizations.	Depth 3	Resource utilization; identify shared assets; license compliance
Spreadsheet Grid View	The KLNK link graph rendered as a spreadsheet sheet (SHT-031 Link Network Sheet): each row is a LinkEdge, columns are from/to node details, edge type, weight, visibility, and shadow row status.	N/A	Data export; bulk link management; programmatic access

5.5 ShadowRow Protocol

ShadowRows are the mechanism by which linked portfolio components from other users appear as rows in your own spreadsheet. They are one of the most powerful features of the KLNK system — enabling cross-portfolio visibility, shared project tracking, and investment monitoring without requiring full data replication.

5.5.1 ShadowRow Lifecycle

1. **Link Created:** When a LinkEdge is created between User A's component and User B's component (e.g. A contracts with B), the system evaluates whether a ShadowRow should be created in B's spreadsheet based on the edge type and B's ShadowRowPolicy.
2. **Negotiation:** If the edge type requires both parties' consent for ShadowRow creation (e.g. Collaborates), an invitation is sent to B. B configures which columns of their component are visible in A's ShadowRow.
3. **Provisioning:** ShadowRow is created in the target spreadsheet's registry with `source_component_id` pointing to A's component. The ShadowRow is marked as read-only.
4. **Sync:** On every update to the source component, a ShadowUpdateEvent is published. The target spreadsheet receives the event and updates the ShadowRow's MirrorColumn values according to the configured update policy (RealTime | Batched | Manual).
5. **Visibility:** The ShadowRow appears in relevant sheets (e.g. Shared Portfolios, Contacts, Contracts) alongside the host user's own rows, clearly marked with a "linked" indicator and the source identity's `@handle`.
6. **Disconnection:** When the LinkEdge is removed, the ShadowRow transitions to `status=Disconnected`. The host user can choose to Archive it (preserve history) or Delete it (remove from spreadsheet). Historical data is always preserved in the EventLog.

ShadowRow Property	Description
<code>source_component_id</code>	ComponentId of the source component in the source user's spreadsheet
<code>source_owner_identity</code>	@handle of the source identity
<code>visible_columns</code>	Vec<ColumnId> — only these columns are populated in the ShadowRow. Other columns are null.
<code>update_policy</code>	RealTime Batched(every_n_hours) Manual — how often MirrorColumns are refreshed from source
<code>write_back_columns</code>	Vec<ColumnId> — columns the host user can write to, which are then synced back to the source (e.g. for comment annotations on contracts)
<code>shadow_status</code>	Active Pending Disconnected Archived — lifecycle state
<code>last_synced</code>	DateTime — timestamp of last successful sync from source
<code>sync_error</code>	Option<String> — last sync error if status is degraded

5.6 Link Discovery & Graph Traversal API

The KLNK system exposes a rich query API for traversing, filtering, and analyzing the link graph. All queries are subject to the VisibilityMask of the target nodes — no query can see nodes or edges that the caller is not permitted to access.

API Endpoint	Method	Description
/klnk/nodes/{node_id}/tree	GET	Get LinkTree rooted at node_id with configurable depth, edge_type filter, and direction (outbound inbound both)
/klnk/nodes/{node_id}/forest	GET	Get LinkForest for all nodes owned by the same entity as node_id
/klnk/nodes/{node_id}/ego	GET	Get ego network (depth 1+2) centered on node_id — all direct and second-degree connections
/klnk/edges	POST	Create a new LinkEdge (link request). Subject to permission checks and edge-type consent requirements.
/klnk/edges/{edge_id}	DELETE	Remove a LinkEdge. Triggers ShadowRow disconnection protocol.
/klnk/shadow-rows	GET	List all ShadowRows in the caller's spreadsheet — linked rows from other users' portfolios
/klnk/shadow-rows/{shadow_id}/sync	POST	Manually trigger a sync of a ShadowRow's mirror columns from source
/klnk/discover	GET	Discover potential link targets: users, components, and orgs matching a query, filtered by visibility and ranked by match_score
/klnk/graph/path	GET	Find shortest path between two nodes in the link graph (Dijkstra with edge weight). Used for "degrees of separation" queries.
/klnk/graph/clusters	GET	Identify community clusters in the link graph (Louvain community detection). Used for org and ecosystem analysis.
/klnk/graph/centrality	GET	Compute betweenness, closeness, and eigenvector centrality for nodes. Used for influence scoring.
/klnk/subscriptions	GET/POST/DELETE	Manage link change subscriptions — receive webhooks or Kafka events when linked nodes change

6. Sheet System — Derived Sheet Taxonomy

A Sheet is a named, scoped, materialized view of the PortfolioComponent registry. Every sheet is defined by: a SheetDefinition (filter predicates, visible columns, default sort, default grouping), a ColumnSchema (the ordered set of columns visible on this sheet), and an optional ComputedColumnSet (additional engine-derived columns rendered only on this sheet). Sheets are not separate databases — they are views over the single underlying PortfolioRow store.

6.1 Master Sheet Registry — All 35 Sheets

Sheet ID	Sheet Name	Primary Row Types	Default Group By	Key Use Case
SHT-001	Master Registry	All types	Type then Status	Root view of all components search and bulk operations
SHT-002	Portfolio Hierarchy	Portfolio, SubPortfolio, Program, Project	Hierarchy (tree)	Visualize structural tree with metrics
SHT-003	Programs	Program	Status	Strategic initiative tracking and alignment
SHT-004	Projects	Project	Program → Status	Operational project management methodology and sprint data
SHT-005	Tasks & Backlog	Task, Story, Epic, Feature	Project → Sprint	Work execution: backlog, sprint, kanban
SHT-006	Resources	Resource	Resource Type	Manage human, machine, li service resources
SHT-007	Assets	Asset	Asset Type	Capital assets, IP, financial & digital assets
SHT-008	Artifacts	Artifact	Project → Artifact Type	Versioned outputs: documents, designs, code, reports
SHT-009	Finances	All financial rows	Quarter → Category	Unified P&L: income, expenses across all engagements
SHT-010	Budget Tracker	Program, Project, Gig, Contract	Program → Status	Budget allocation, spend, revenue and utilization
SHT-011	Work & Gigs	Gig, Contract, Job, Task	Platform → Status	All income-generating engagements across all platforms
SHT-012	Deliverables	Project, Artifact, Release	Project → Status	Output tracking: what was produced and when
SHT-013	Timeline	All dated rows	Quarter → Program	Temporal view of all work with milestones
SHT-014	Roadmap	Program, Project, Milestone	Program → Quarter	Strategic roadmap for external
SHT-015	Portable Benefits	BenefitAccount	Benefit Type	Benefits dashboard: balance, contributions, coverage gaps

Sheet ID	Sheet Name	Primary Row Types	Default Group By	Key Use Case
SHT-016	Grants & Microfinancing	Grant, Campaign (grant type)	Status	Grant pipeline from discovery to disbursement and reporting
SHT-017	Equity & Crowdfunding	Campaign, Investment	Campaign Type	Fundraising, equity rounds, investment positions
SHT-018	Group Economics	Shared Portfolio, Revenue Pool, Org	Organization	Cooperative and collective financial structures
SHT-019	Organizations	Profile (org type)	Org Type	All organizations, collectives, cooperatives the user is part of
SHT-020	Shared Portfolios	Portfolio (shared)	Owner Org	Co-owned and shared portfolio components
SHT-021	Collaboration	All shared/contributed rows	Contributor	Contribution tracking and attribution weights
SHT-022	Risk Register	All rows with risk_flags	Risk Severity	Unified risk dashboard across components
SHT-023	Analytics Dashboard	All rows	Type → Health Score	Health scores, performance signals
SHT-024	Feed & Community	Profile, Post, Artifact (public)	Space	Community-facing content and activity
SHT-025	Contacts & Network	Profile, Contact	Relationship Type	CRM: client, collaborator, investor, vendor contacts
SHT-026	Marketplace Listings	Asset, Resource, Gig (public)	Category	Published marketplace items and performance
SHT-027	Exchange	Investment, Deal, Campaign, Asset	Exchange Type	Exchange market activity: bonds, instruments
SHT-028	Archive	All archived rows	Archive Date	Deep storage with full restoration
SHT-029	Event Log	EventLog entries (read-only)	Date → Actor	Immutable audit trail of all mutations
SHT-030	Snapshots	Snapshot records	Date	Point-in-time portfolio snapshots for restore
SHT-031	Link Network	LinkEdge records	Edge Type → Target	All inter-portfolio link connections, KLNK graph in spreadsheet
SHT-032	Identity & Profiles	ProfilePartition records	Identity	Multi-identity management: visibility masks, account management
SHT-033	Portable Benefits Deep Dive	BenefitAccount + Gig (contribution)	Benefit Type → Platform	Gig platform contribution tracking and benefit projection
SHT-034	Custom Sheet (User)	User-defined	User-defined	Ad-hoc views for specific work, client reports
SHT-035	Template Library	ItemBook:Template rows	Category	Reusable templates available in KOGI-APPSTORE and user library

6.2 SHT-001: Master Registry Sheet

The Master Registry is the root spreadsheet — the unfiltered, full-column view of every PortfolioComponent the authenticated user can access. It is the entry point for global search, bulk operations, cross-type analytics, and administrative management. Every other sheet is a subset or derived view of the Master Registry.

Configuration Element	Value
Pinned Columns (always visible)	component_id · slug · component_type · item_type · name · status · visibility · owners
Default Visible Columns	+ created_at · updated_at · progress_pct · health_score · risk_score · due_date · budget_remaining · tags · link_count
Optionally Visible	All remaining 150+ columns — user-toggable per session or saved view
Default Sort	updated_at descending
Default Group	component_type
Row Actions	Open · Edit · Duplicate · Archive · Share · Export · Add to Sheet · Pin · Set Status · Link to Network
Bulk Actions	Set Status · Set Visibility · Assign Tags · Archive · Export Selected · Add to Binder · Link to Org · Tag Identity
AI Overlay	Oba surfaces: anomalies, stale items (not updated >30 days), health warnings, overdue items, missing coverage, and quick-action suggestions inline as row-level annotations
Link Network	link_count column shows inter-portfolio connections. Click to open Component Tree View in KLNK.

6.3 SHT-004: Projects Sheet — Detailed Column Schema

The Projects Sheet is the primary operational view for project-level work. It exposes the full ProjectPayload alongside financial, governance, AI, and link network columns.

Column Group	Columns	Notes
Identity	component_id · name · slug · color · icon	Standard identity group
Classification	project_type · methodology · domain · tags	ProjectType: Software Design Content Research Operations Event Custom
Lifecycle	status · state · lifecycle_stage · visibility	Full lifecycle state machine with Oba annotations on transitions

Column Group	Columns	Notes
Ownership	owners · editors · watchers	Multi-user role columns; avatar cluster display
Timeline	start_date · end_date · due_date · progress_pct · schedule_variance_days · predicted_completion_date	Key scheduling columns + AI prediction
Sprints	sprint_count · current_sprint · sprint_velocity · backlog_size · throughput	Derived from ProjectPayload.sprints; auto-updated on sprint completion
Releases	release_count · latest_release · next_release_date	Derived from ProjectPayload.releases
Finance	budget_allocated · budget_spent · budget_remaining · budget_utilization_pct · revenue · profit · roi	Full financial tracking; kogi-bank synced
Governance	governance_model · approval_status · risk_flags · compliance_flags · policy_ids	Governance state with pending approval count badge
Analytics	health_score · risk_score · cycle_time_days · lead_time_days · velocity · collaboration_score · anomaly_flags	AI-computed performance signals; all from kogi-engine
Work & Delivery	story_type · priority · story_points · open_task_count · completed_task_count · acceptance_criteria_count	Work execution metrics
AI Signals	predicted_completion_date · schedule_risk_flag · budget_risk_flag · oba_hint · match_score	Engine-generated forward-looking signals
Link Network	link_count · inbound_link_count · linked_portfolio_ids · shadow_row_count	Cross-portfolio connections for this project
ToolBoxes	toolbox_ids · toolbox_names	Attached TMS ToolBoxes

6.4 SHT-009: Finances Sheet — Extended Column Schema

The Finances Sheet is a unified financial ledger view spanning all portfolio components that carry financial data. It aggregates income, expenses, budget, and asset values across gigs, contracts, projects, campaigns, benefits, and investments. It is the master financial spreadsheet — the equivalent of a real-time P&L across the worker's entire portfolio of work.

Column	Type	Applies To	Notes
name	String	All	Component display name
component_type	ComponentType	All	Row type for period grouping

Column	Type	Applies To	Notes
quarter	String	All	Fiscal quarter label (e.g. Q1-2026)
fiscal_year	u32	All	Fiscal year for budget alignment
revenue	Decimal	Gig, Contract, Job, Campaign, Investment	Income or revenue generated; kogi-bank synced
expenses	Decimal	Project, Program, Asset, Gig	Costs incurred; kogi-bank synced
budget_allocated	Decimal	Program, Project	Planned budget envelope
budget_spent	Decimal	Program, Project	Actual spend to date
budget_remaining	ComputedColumn	Program, Project	= allocated - spent
profit	ComputedColumn	All	= revenue - expenses
roi	ComputedColumn	Asset, Investment, Campaign	Return on investment %
tax_category	TaxCategory	All	Business SelfEmployment Investment Benefit — for tax report generation
invoice_refs	Vec<InvoiceId>	Contract, Gig, Job	Badge showing count; click to expand invoice list
payment_status	PaymentStatus	Contract, Gig, Job	Unpaid Partial Paid Overdue Disputed — color-coded badge
gig_earnings	Decimal	Gig rows	Base earnings before tips and deductions
tips	Decimal	Gig rows	Tip income; displayed separately for platform contribution calculation
portable_contribution	Decimal	Gig rows	Platform contribution to portable savings (e.g. 4% of pre-tip)
mileage	Option<Decimal>	Gig rows	Miles driven; used for IRS mileage deduction calculation
mileage_deduction	ComputedColumn	Gig rows	= mileage x current IRS rate per mile
benefit_contributions	Decimal	BenefitAccount rows	Cumulative contributions to benefit account YTD
benefit_balance	Decimal	BenefitAccount rows	Current account balance from kogi-bank

Column	Type	Applies To	Notes
investment_value	Decimal	Investment, Asset rows	Current estimated valuation
distributions_received	Decimal	Investment rows	Cumulative distributions received
grant_awarded	Decimal	Grant rows	Total grant award amount
grant_disbursed	Decimal	Grant rows	Amount disbursed to date
campaign_target	Decimal	Campaign rows	Fundraising target amount
campaign_raised	Decimal	Campaign rows	Amount raised to date
campaign_pct	ComputedColumn	Campaign rows	= raised / target x 100
bank_account_ref	AccountId	All	Linked kogi-bank account

6.5 SHT-015: Portable Benefits Sheet — Full Column Schema

Every portable benefit account is a PortfolioItem (kind: BenefitAccount) and surfaces as a row on this sheet. This is the worker's complete, real-time benefits dashboard — balances, contributions, coverage gaps, and tax optimization signals all in one place.

Column	Type	Benefit Types	Notes
name	String	All	Account display name
benefit_type	BenefitType	All	Health Dental Vision HSA Retirement_SEP Retirement_401k Retirement_PEP PTO Disability WorkersComp ProfessionalDev EmergencySavings PortableSavings IncomeProtection
provider	String	All	Benefit provider name (e.g. Stride LLC, Fidelity, Cigna, TIAA)
status	ComponentStatus	All	Active Inactive Suspended Archived
balance	Decimal	HSA, Retirement, Emergency, Portable, ProfDev	Current account balance from kogi-bank
ytd_contributions	Decimal	HSA, Retirement, Portable	Year-to-date contributions from all sources

Column	Type	Benefit Types	Notes
contribution_sources	Vec<Contribution Source>	Portable	Gig platforms contributing (e.g. DoorDash 4%, Lyft 3%) with amounts
platform_contribution_ytd	Decimal	Portable	Total gig platform contributions YTD
self_contribution_ytd	Decimal	All	User-directed contributions YTD
vesting_pct	ComputedColumn	Retirement	Vesting schedule completion percentage; cliff and schedule applied
projected_balance_1yr	ComputedColumn	Retirement, Portable	Engine-projected balance at 1 year given current contribution rate
projected_balance_5yr	ComputedColumn	Retirement	Engine-projected balance at 5 years (compounding model)
coverage_gap_flag	AnomalyFlag	All	BenefitsEngine flag: coverage gap detected — type + recommended action
pto_accrued_days	Decimal	PTO	Current accrued paid time off in days
pto_used_ytd_days	Decimal	PTO	PTO used year-to-date
pto_remaining_days	ComputedColumn	PTO	= accrued - used
income_replacement_days	ComputedColumn	Disability, PTO, IncomeProtection	Coverage days vs. income gap risk — how many days of income are covered
tax_treatment	TaxCategory	All	PreTax PostTax TaxFree — affects tax savings computation
tax_savings_ytd	ComputedColumn	HSA, Retirement	Estimated tax savings from pre-tax contributions YTD
irs_contribution_limit	ComputedColumn	HSA, Retirement	Current IRS annual contribution limit for this benefit type
remaining_contribution_room	ComputedColumn	HSA, Retirement	= irs_contribution_limit - ytd_contributions
claim_count	u32	Health, Disability	Number of claims filed this year

Column	Type	Benefit Types	Notes
linked_bank_account	AccountId	All	kogi-bank BenefitAccount linked to this portfolio item

6.6 SHT-031: Link Network Sheet

The Link Network Sheet is the spreadsheet representation of the KLNK graph — every inter-portfolio link the user has, presented as a queryable, filterable, exportable table of rows. Each row is a LinkEdge.

Column	Type	Description
edge_id	EdgeId	Unique identifier for this link edge
edge_type	LinkEdgeType	Follows Subscribes Watches Collaborates InvestedIn Donated Contracted DependsOn ResourceShares FederationPeer OrgMembership Mentors Endorses References
direction	ComputedColumn	Outbound (I linked to them) Inbound (they linked to me) Mutual
from_node_name	String	Display name of the source node (my component or identity)
from_node_type	LinkNodeType	PortfolioComponent Profile Identity Organization Portfolio FederationNode
to_node_name	String	Display name of the target node (their component or identity)
to_node_owner_handle	String	@handle of the user/identity who owns the target node
to_node_type	LinkNodeType	Type of the target node
visibility	LinkVisibility	How visible this link is: Public Followers Connections Private
shadow_row_id	Option<UUID>	If a ShadowRow was created from this link, its ID. Click to navigate to the shadow row.
shadow_status	Option<ShadowStatus>	Active Pending Disconnected — state of the shadow row if one exists
last_synced	Option<DateTime>	Last time the shadow row was synced from source
weight	f64	Edge weight in graph algorithms (0.0-1.0). Higher = stronger connection.
created_at	DateTime	When this link was established

Column	Type	Description
link_depth	ComputedColumn	Shortest path length to this node from the user's root identity
mutual_connections	ComputedColumn	Count of mutual connections (shared nodes in both users' link graphs)

7. View Engine (KPWW)

7.1 ViewDefinition Schema

The View Engine transforms raw PortfolioRow data into the shaped, filtered, sorted, grouped, and visually enriched spreadsheet that the user interacts with. Views are defined as ViewDefinition objects, serialized to JSON, versioned, and shareable.

Field	Type	Description
id	UUID	View identifier
name	String	User-defined view name
sheet_id	SheetId	Parent sheet this view belongs to (or "master" for cross-sheet views)
base_row_filter	Option<ViewFilter>	Pre-filter applied before any user interaction (e.g. "only rows owned by me")
column_schema	ColumnSchema	Ordered list of visible columns with width, pin, and sort settings per column
filters	Vec<ViewFilter>	User-configurable filter predicates. Evaluated as AND within a group, OR across groups.
sorts	Vec<ViewSort>	Ordered sort predicates. Primary sort first, secondary second, etc.
groups	Vec<ViewGroup>	Hierarchical grouping definitions. First group is outermost heading row.
row_height	RowHeight	Compact Normal Tall Auto — controls card density in list view
highlight_rules	Vec<RowHighlightRule>	Conditional color rules. E.g. health_score < 60 → red background
pinned_columns	Vec<ColumnId>	Columns always visible regardless of horizontal scroll position
frozen_row_count	u32	Number of rows pinned at top (e.g. summary or header rows)
pivot_config	Option<PivotConfig>	Pivot table mode configuration: row_field, col_field, value_field, aggregation
chart_config	Option<ChartConfig>	Chart overlay: type (bar/line/pie/gauge), x_column, y_column, series
summary_row	Option<SummaryRowConfig>	Aggregate row at sheet bottom: sum/avg/count/max/min per selected numeric columns
board_config	Option<BoardConfig>	Kanban/Gantt/Calendar board mode settings
link_network_overlay	bool	When true, show link_count badges on rows and allow click-to-open KLNK tree view

Field	Type	Description
identity_filter	Option<IdentityTag>	Filter to only show rows tagged to a specific identity/profile partition
visibility	ViewVisibility	Private Shared Public Template — governs who can see and copy this view
created_by / at / updated_at	UserId / DateTime	Authorship and timestamps

7.2 Filter System

Filters are composable boolean predicates evaluated against row columns. Complex filter trees can be built by nesting AND and OR groups.

Filter Type	Syntax	Example
Equality	column = value	status = Active
Inequality	column != value	visibility != Private
Numeric Range	column BETWEEN a AND b	health_score BETWEEN 60 AND 100
Greater/Less	column > or < value	budget_utilization_pct > 80
Contains (string)	column CONTAINS substring	name CONTAINS "API"
In Set	column IN [v1, v2, ...]	item_type IN [Project, Program]
Not In Set	column NOT IN [v1, ...]	status NOT IN [Archived, Cancelled]
Date Range	column AFTER / BEFORE / BETWEEN dates	due_date AFTER 2026-03-01
Is Empty	column IS EMPTY	due_date IS EMPTY
Is Not Empty	column IS NOT EMPTY	owners IS NOT EMPTY
Tag Includes	tags INCLUDES tag	tags INCLUDES "q2-initiative"
Owner Is Me	owners INCLUDES me	owners INCLUDES me
Identity Tagged	identity_tags INCLUDES identity	identity_tags INCLUDES "@jordan-dev"
Has Shadow Row	shadow_row_count > 0	Shows only rows visible to linked portfolios
Has Link	linked_portfolio_ids IS NOT EMPTY	Shows only rows with active inter-portfolio links
Engine Score	health_score >= threshold	health_score >= 70
Has Flag	risk_flags HAS_SEVERITY HIGH	risk_flags HAS_SEVERITY HIGH

Filter Type	Syntax	Example
AND / OR Groups	(filter1 AND filter2) OR filter3	(status=Active AND health_score < 60) OR risk_score > 80

7.3 Board Modes

Every sheet can be rendered in an alternative board mode. Board modes are view-layer transforms — the underlying PortfolioRow data is unchanged; only the rendering changes.

Board Mode	Configuration	Best For
Kanban Board	Columns = status values; cards show name, owner, due_date, health_score; WIP limits configurable per column	Task and project status management; visual WIP management
Gantt Chart	Rows = timeline components; horizontal bars = start_date to end_date; dependencies rendered as arrows; critical path highlighted	Project and program timeline planning; dependency management
Calendar	Rows with date columns placed on calendar grid by due_date or start_date; month/week/day views	Scheduling, deadlines, milestone tracking, booking management
Agile Board	Sprint-scoped Kanban with story point totals per column; burndown chart overlay; sprint velocity badge	Sprint planning and execution; Scrum and Kanban workflows
Resource Board	Rows = resources; columns = time periods; cells = allocation %; red highlighting for over-allocation	Capacity planning and conflict detection (AllocationEngine)
Network Graph	Nodes = rows; edges = GraphEdges; force-directed layout; color by type; size by health_score	Dependency visualization; portfolio relationship map; KLNK exploration
Treemap	Hierarchical tiles sized by a numeric column (e.g. budget_allocated); color by health_score; drill-down	Portfolio composition and budget distribution at a glance
Timeline Board	Swimlane rows = programs; bars = projects and milestones; nested structure; quarter markers	Cross-program roadmap view; strategic planning overview
Link Forest View	Nodes = portfolio components and linked external nodes; tree structure by identity; KLNK rendering	Visualize all cross-portfolio connections; discover network structure
Identity Grid	Rows grouped by identity_tag; columns show partition visibility settings; color by isolation_level	Multi-identity management; profile visibility configuration

8. Column System — Full Type Registry

8.1 Column Types

Column Type	Description	Storage	Aggregations
TextField	Free text string; sortable, searchable, filterable; supports markdown preview in tall row mode	String, FTS-indexed by SearchEngine	COUNT, COUNT_NON_EMPTY
NumberField	Integer or decimal; supports all numeric aggregations; renders with configurable decimal places and thousands separators	Decimal, range-indexed	SUM, AVG, MIN, MAX, MEDIAN, COUNT
CurrencyField	Decimal with ISO 4217 currency code; multi-currency aware; auto-conversion to user base currency for aggregation	Decimal + CurrencyCode	SUM, AVG, MIN, MAX (in base currency)
PercentField	Decimal 0-100; renders with % suffix; progress bar visualization in tall row mode; sortable and range-filterable	f32	AVG, MIN, MAX, MEDIAN
DateField	Calendar date; timezone-aware; relative display ("in 3 days", "2 weeks ago"); supports date math in formulas	Date, calendar-indexed	MIN (earliest), MAX (latest), RANGE
DateTimeField	Full timestamp; nanosecond precision for EventLog entries; timezone-aware; relative or absolute rendering	DateTime<Utc>	MIN, MAX, RANGE
DurationField	Time span; rendered as "3d 4h"; supports arithmetic; auto-computed from date range columns	Duration (seconds)	SUM, AVG, MIN, MAX
EnumField	One-of a defined value set; renders as colored badge; fast equality filter; configurable badge colors per value	String enum, hash-indexed	COUNT_BY_VALUE, MODE
MultiEnumField	Set of enum values; renders as multiple badge chips; "includes" filter; max configurable	Vec<String>, set-indexed	COUNT_BY_VALUE, INTERSECTION
RelationField	Foreign key reference to another PortfolioRow; renders as linked name chip; click-to-navigate; cross-sheet joins	ComponentId, join-resolved at query time	COUNT, COUNT_UNIQUE
MultiRelationField	Set of foreign key references; renders as linked name chip cluster; max 50 relations by default	Vec<ComponentId>	COUNT, COUNT_UNIQUE

Column Type	Description	Storage	Aggregations
UserField	Reference to a user or organization entity; resolves to display name + avatar; @mention trigger in cell edit	EntityId	COUNT_UNIQUE
MultiUserField	Set of user/org references; renders as avatar cluster ("N" for overflow); filterable by specific user	Vec<EntityId>	COUNT_UNIQUE, LIST
TagField	Vec<String>; renders as tag chips; "includes" filter; tag autocomplete; global tag taxonomy	Vec<String>, tag-indexed	UNION, INTERSECTION, COUNT_BY_TAG
BoolField	Boolean; renders as checkbox; filterable; tri-state in some contexts (True/False/Null)	bool	COUNT_TRUE, COUNT_FALSE, PCT_TRUE
RichTextField	Markdown-formatted rich text; not sortable; searchable; preview mode in card views	Markdown String	COUNT (non-empty)
FileRefField	Reference to a file or artifact stored in kogi-content; renders with file type icon and preview on hover	FileId, resolves to metadata + URL	COUNT
URLField	Web URL; renders as clickable link with domain favicon; link preview on hover	String	COUNT
ComputedColumn	Derived value computed by Computation Engine from other columns; read-only; recalculated on dependency change	Computed at query time; cached with TTL	All numeric aggregations
AIColumn	Engine-generated signal: score, flag, recommendation, prediction; read-only; written by kogi-engine via plugin writeback	Written by kogi-engine; cached; TTL-invalidated on EventLog events	AVG, MIN, MAX, MEDIAN
FormulaColumn	User-defined formula over other columns using ColumnComputer expression language; power user feature	Evaluated by ColumnComputer; cached	All numeric aggregations
AuditColumn	Derived from EventLog; shows last actor, last action, action count, creation info	Derived from EventLog at query time	COUNT
AnalyticsColumn	Aggregated engagement or performance metric (view count, click count, engagement rate); updated by AnalyticsEngine	Updated by AnalyticsEngine; stored in materialized view	SUM, AVG, MAX

Column Type	Description	Storage	Aggregations
LinkNetworkColumn	Computed from KLNK GraphEdge store: link_count, linked_identities, shadow_row_count, link_depth	Computed from GraphEdge store; cached	SUM, AVG, MAX, COUNT
IdentityColumn	Identity-specific metadata: which identity tags apply to this row, which profiles it appears on	Vec<IdentityTag> from ProfilePartition store	COUNT, COUNT_BY_TAG

8.2 Formula Engine — ColumnComputer

The ColumnComputer evaluates FormulaColumn definitions at query time. Formulas are defined in a type-safe expression language supporting the following operations:

Function Category	Functions	Example
Arithmetic	+ - × ÷ % ^ ABS ROUND CEIL FLOOR	profit = revenue - expenses
Comparison	= != > < >= <=	budget_utilization_pct > 80
Logical	AND OR NOT IF(cond, true_val, false_val) SWITCH CASE	IF(health_score < 60, "At Risk", "On Track")
String	CONCAT UPPER LOWER LEN CONTAINS STARTSWITH TRIM LEFT RIGHT MID	CONCAT(name, " - ", status)
Date	TODAY DAYS_BETWEEN DATE_ADD QUARTER YEAR MONTH DAY WEEKNUM	DAYS_BETWEEN(start_date, due_date)
Aggregation	SUM AVG COUNT MAX MIN MEDIAN (over related children)	SUM(CHILDREN.budget_spent)
Lookup	RELATED(relation_col, target_col) LOOKUP(key, table, col)	RELATED(program_ref, budget_allocated)
Engine	HEALTH_SCORE() RISK_SCORE() MATCH_SCORE() INCOME_PROJECTION(days)	HEALTH_SCORE() >= 80
Identity	IDENTITY_TAG() IS_IN_PARTITION(tag) VISIBLE_TO(observer_type)	IS_IN_PARTITION("@jordan-dev")
Link Network	LINK_COUNT() INBOUND_LINKS() OUTBOUND_LINKS() IS_LINKED_TO(portfolio_id)	LINK_COUNT() > 5

8.3 Custom Columns

Feature	Description
Add Column	Any user with Editor permission on the sheet can add a custom column of any supported type
Column Scope	Custom columns can be scoped to a single sheet (local) or promoted to all sheets (global) by an owner
Default Values	Custom columns support default values, required flags, and validation rules (min/max, regex, enum set, conditional default)
Computed Custom Column	A FormulaColumn built from other columns using the formula editor (arithmetic, conditional, lookup, engine functions)
Org Templates	Organizations can define standard column sets as ColumnTemplates, applied to all member portfolios via org governance
Identity-Scoped Columns	Custom columns can be scoped to a specific identity partition — visible only when viewing that identity's profile
API Access	Custom column values accessible via kogi-dev API under <code>component.data.properties["column_key"]</code>
Export	Custom columns appear in all export formats (CSV, XLSX, JSON) with user-defined column headers
Shadow Column Mirroring	Custom columns on a source component can be mirrored as MirrorColumns in a linked user's ShadowRow, subject to visibility policy

9. Computation Engine (KPCM)

The Computation Engine powers all ComputedColumns, AIColumns, and rollup aggregations in the spreadsheet. It runs on two tiers: synchronous in-request computation for simple derived columns (arithmetic, rollup), and asynchronous kogi-engine computation for complex AI-driven signals (health score, risk score, match score, predictions). All computed results are cached in the CellStore with a staleness TTL and invalidated on relevant EventLog events.

9.1 All Computational Models

Model	Inputs	Output	Consumer Sheets
PortfolioHealth	child component statuses, budget utilization, risk flags, progress %, active owner count, engagement, collaboration_score, benefit coverage	health_score 0-100 + dimension breakdown	SHT-001, SHT-002, SHT-023
ProjectMetrics	sprint velocity history, backlog size, completion rate, cycle time, lead time, blocker count	completion_pct, velocity, risk_level, predicted_completion_date	SHT-004, SHT-023
ProgramAlignment	child project health scores, KPI actuals vs targets, budget rollup, milestone status	alignment_score 0-100, KPI_achievement_rate	SHT-003, SHT-002
SubPortfolioRollup	all descendant component financial and status data via recursive tree traversal	rollup_score 0-100, aggregated financials	SHT-002, SHT-023
ResourceUtilisation	resource allocation records, capacity units, consumed units, schedule overlap	utilisation_pct, demand_ratio, over_committed bool	SHT-006, SHT-010
AssetValue	asset type, acquisition cost, valuation history, market data (if available), depreciation schedule	current_value, depreciation, roi_pct, irr	SHT-007, SHT-009, SHT-027
ArtifactMaturity	artifact type, version count, review status, linked project completion, documentation completeness	maturity_score 0-100, completeness breakdown	SHT-008, SHT-012
BinderCoverage	expected item IDs (from template), actual item IDs present	coverage_pct, missing_count, extra_count	SHT-001 (Binder rows)

Model	Inputs	Output	Consumer Sheets
BookConsistency	section count, empty sections, schema compliance count per book type	consistency_score 0-100	SHT-001 (Book rows)
FolderOrganisation	max depth, orphan count, duplicate names, naming convention compliance	organisation_score 0-100	SHT-001 (Folder rows)
RecordIntegrity	entry count, duplicate count, integrity hash presence, required fields	integrity_score 0-100	SHT-001 (Record rows)
CollaborationScore	contributor count, contribution velocity, governance participation, merge conflict rate, attribution weight distribution	collaboration_score 0-100	SHT-020, SHT-021, SHT-023
BenefitsHealth	benefit account balances, coverage types present vs required, income replacement ratio, vesting progress	benefits_health_score 0-100, coverage_gap list	SHT-015, SHT-023
RiskScore	risk_flags (severity + probability), mitigation status, overdue items, blocker chain depth	risk_score 0-100, top_risk_items	SHT-022, SHT-023, all sheets
IncomeProjection	gig earnings history (90d), active contract revenue, investment distributions, benefit contributions, seasonal patterns	projected_income_{30,60,90}d with confidence intervals	SHT-009, SHT-023
CampaignHealth	contribution velocity, time-to-deadline, backers-per-day, social graph engagement with campaign	campaign_health_score, success_probability, days_to_goal	SHT-017, SHT-016
GrantMatchScore	eligibility criteria match, portfolio strength relative to grant requirements, historical award data for similar portfolios	grant_match_score 0-100, eligibility_flags	SHT-016
LinkNetworkStrength	link_count, inbound_link_count, link_diversity (edge types), mutual connection count, network centrality	network_strength_score 0-100, centrality_rank	SHT-031, SHT-025
IdentityCoverageScore	completeness of each ProfilePartition: required fields, visibility mask set, public content present	identity_coverage_score per partition	SHT-032

Model	Inputs	Output	Consumer Sheets
MarketabilityScore	public asset listing quality (title, description, media, reviews), pricing competitiveness vs PriceEngine, CTR/conversion	marketability_score, listing_optimization_hints	SHT-026

9.2 PortfolioHealth Scoring — Full Algorithm

The PortfolioHealth model is the master analytical output of the Computation Engine for Portfolio and SubPortfolio rows. It computes an overall health_score (0-100) from a weighted composition of seven sub-dimensions.

Dimension	Weight	Inputs	Scoring Logic
Delivery Health	25%	progress_pct, schedule_variance_days, milestone completion rate, predicted_completion vs due_date	Full score if on-track; decreasing score for schedule variance; zero for abandoned milestones
Financial Health	20%	budget_utilization_pct, cash_flow signal, ROI trend, overrun_flags, outstanding_invoices	Full score if utilization < 80%; decreasing for overruns; penalty for unpaid overdue invoices
Risk Health	20%	risk_score, open_risk_count, critical_risk_count, mitigation_completion_rate	Inverse of risk_score; full if no open critical risks; rapid decay with unmitigated high-severity risks
Resource Health	15%	resource utilization_pct, over_allocation_flags, toolbox coverage, skill gap flags	Full score if utilization 60-80%; penalty for over-allocation (>100%) and under-allocation (<30%)
Engagement Health	10%	engagement_rate, follower_growth, active_contributor_count, collaboration_score, link_count growth	Measures portfolio visibility, community engagement, and network growth; relevant for public/shared portfolios
Governance Health	5%	open_approval_requests, compliance_flags, overdue_reviews, charter_present, policy_ids_attached	Full score if all approvals resolved; decreasing for unresolved compliance flags
Benefit Coverage Health	5%	benefits_health_score, coverage_gap_flags, pto_remaining_days, income_replacement_ratio	Relevant for worker's personal portfolio; scores adequacy of portable benefits coverage

9.3 Aggregation Rules

Column Type	Aggregations	Notes
NumberField / CurrencyField / PercentField	SUM · AVG · MIN · MAX · MEDIAN · COUNT · COUNT_NON_EMPTY	SUM is default for financial columns; AVG for percentage/score columns
DateField / DateTimeField	MIN (earliest) · MAX (latest) · RANGE (latest - earliest)	Useful for "earliest deadline in group" or "project span"
EnumField	COUNT_BY_VALUE (distribution) · MODE (most common)	Rendered as mini bar chart in group row
TagField	UNION (all tags) · INTERSECTION (shared tags) · COUNT_BY_TAG	Displayed as tag cloud in group header
RelationField / MultiRelationField	COUNT · COUNT_UNIQUE	Count of related rows
UserField / MultiUserField	COUNT_UNIQUE · LIST	Count of distinct users; list as avatar cluster
BoolField	COUNT_TRUE · COUNT_FALSE · PCT_TRUE	E.g. "7 / 12 tasks complete (58%)"
ComputedColumn / AIColumn	AVG · MIN · MAX · MEDIAN	Aggregate the computed signal across group members
LinkNetworkColumn	SUM · AVG · MAX	Total links, average link depth, most-connected component
IdentityColumn	COUNT · COUNT_BY_TAG	How many rows per identity partition

10. Collaborative Editing (KPCL)

10.1 CRDT Architecture

The Portfolio Spreadsheet is a live, multi-user, collaboratively editable document. Multiple users — across multiple devices, time zones, and federation nodes — can simultaneously view and edit the same portfolio. Consistency is maintained by the CRDT engine (LWW field writes + OR-Set for set-valued columns), and every mutation is appended to the immutable EventLog.

For the multi-identity case, CRDT operations are tagged with both the `node_id` (physical source) and the `identity_tag` (logical source), enabling conflict resolution to respect identity-partition boundaries. An edit to a row tagged to Identity A by Identity B is treated as a cross-partition edit and may require approval depending on the partition's write policy.

10.2 CRDT Operation Types

CRDT Operation	Semantics	Identity Awareness
SetField (LWW)	Last-Write-Wins field update. Write a scalar or enum column value with a VectorClock timestamp. Higher timestamp wins concurrent writes.	<code>actor_identity_tag</code> included in operation. Cross-partition writes checked against SplitPolicy.
AddToSet (OR-Set)	Add a value to a set-valued column (tags, owners, policy_ids, toolbox_ids, identity_tags) with a unique tag. Concurrent adds both survive.	Set additions for <code>identity_tags</code> require write permission on the target partition.
RemoveFromSet (OR-Set)	Remove a specific tagged entry from a set-valued column. Only the explicitly tagged entry is removed; concurrent adds survive.	Identity tag removals are audit-logged separately as PartitionTagEvent.
AppendLog	Append an EventLog entry. Commutative — all concurrent appends survive; ordered by VectorClock causal timestamp.	EventLog entries carry <code>identity_tag</code> for attribution. Cross-partition entries clearly marked.
AttachChild / DetachChild	Add or remove a GraphEdge::Hierarchy edge. Concurrent attaches both survive; concurrent detach resolves to higher-timestamp operation.	Cross-partition hierarchy edges require owner approval if target partition is isolated.
CreateShadowRow	Specialized operation for KLNK: create a ShadowRow in target spreadsheet when a LinkEdge is established.	ShadowRows are always associated with a specific identity's partition. Visibility governed by source's VisibilityMask.
SyncMirrorColumn	Update a MirrorColumn value in a ShadowRow from the source component. Idempotent; last-sync-wins semantics.	Source identity must still have an active LinkEdge to target. Revoked links block further syncs.

CRDT Operation	Semantics	Identity Awareness
StatusTransition	Status column mutations follow the lifecycle state machine. Invalid transitions rejected by PolicyEngine regardless of VectorClock.	Status transitions on shared components may require governance approval depending on GovernanceModel.

10.3 Conflict Resolution

Conflict Type	Resolution Strategy
Concurrent scalar field writes (LWW)	Higher VectorClock timestamp wins. Losing write preserved in EventLog as ConflictRecord for human review. Oba notifies affected users.
Concurrent set additions	Both values survive (OR-Set). No conflict — both are correct. No notification needed.
Concurrent add and remove of same set value	Remove wins if remove's VectorClock dominates; otherwise add wins. ConflictFlag displayed on row.
Concurrent status transitions to incompatible states	Both rejected; component status reverts to last confirmed state. MergeConflict event appended; users notified via KNTF alert.
Concurrent cross-partition writes	Both writes preserved in EventLog. Write that violates SplitPolicy is marked as pending_approval. Owner or admin resolves via governance workflow.
ShadowRow sync failure	ShadowRow marked as shadow_status = Degraded. Last successful sync data preserved. Retry with exponential backoff. User notified after 3 consecutive failures.
CRDT sync failure across federated nodes	Operations buffered in CrdtLog until connectivity restored. Sync replays buffered operations in causal order on reconnect. Conflicts surfaced as review items.
Concurrent contributions to shared portfolio	Both contributions submitted to steward review queue. Steward merges manually or approves both independently. CollaborationEngine recomputes attribution weights.

10.4 Contribution Attribution in Collaborative Sheets

For shared portfolio components and crowdresourced sheets, every row mutation generates a ContributionRecord attached to the component's EventLog. The CollaborationEngine continuously recomputes attribution weights based on contribution history.

Attribution Formula	Description	Default Weight
Labor-based	$\text{weight} = \text{hours_contributed} / \text{total_hours_contributed_by_all}$	60% of composite weight
Equity-based	$\text{weight} = \text{equity_share_pct} / 100$	30% of composite weight

Attribution Formula	Description	Default Weight
Governance Participation	$\text{weight} = \text{votes_cast} / \text{total_votes_in_period}$	10% of composite weight
Hybrid (Cooperative Default)	$\text{weight} = 0.60 \times \text{labor_weight} + 0.30 \times \text{equity_weight} + 0.10 \times \text{governance_weight}$	Default for cooperative entities
Custom Formula	Organization defines formula via GovernanceProposal; CollaborationEngine evaluates against ContributionRecords	Configured per org via PolicyEngine

10.5 Real-Time Collaboration Protocol

Real-time collaborative editing is powered by WebSocket connections managed by the KPCL service. The protocol ensures that all users viewing the same sheet see the same state within 150ms of any mutation.

7. User opens a sheet: client connects to KPCL service via WebSocket. Server sends current sheet snapshot + VectorClock state.
8. User edits a cell: client generates a CrdtOperation locally and applies it optimistically. Operation sent to server over WebSocket.
9. Server receives operation: validates permissions, applies to CrdtLog, broadcasts to all other connected clients on same sheet.
10. Remote clients receive broadcast: apply the operation to their local state. Cursor indicators show other users' active cells.
11. Conflict detected: if a concurrent operation creates a conflict, server generates a MergeConflict record and broadcasts a conflict notification to affected clients.
12. Conflict resolution: user or Oba resolves the conflict. Resolution is a new CrdtOperation that overwrites both conflicting values.
13. Persistence: CrdtLog is flushed to PostgreSQL EventLog asynchronously. Redis CrdtBuffer holds in-flight operations.

11. kogi-engine Integrations

The Portfolio Spreadsheet is both a data source and a display target for the full kogi-engine intelligence stack. Every engine contributes computed columns, derived rows, anomaly flags, and Oba recommendations to the spreadsheet. The integration protocol is bidirectional: the spreadsheet emits EventLog events to the engine; the engine writes computed values back via the plugin writeback protocol.

11.1 Engine Contribution Registry

Engine	Columns Contributed	Writeback Protocol	Update Trigger
AnalyticsEngine	views, clicks, ctr, engagement_rate, followers, spread, share_count, sentiment_score	Plugin writeback: on_component_updated()	On every engagement event; batched for high-frequency events
TelemetryEngine	flow_stats, event_count, last_event_timestamp, mutation_velocity	Plugin writeback: on_event()	Real-time streaming; Kafka consumer
RecommendationEngine	recommendation_score, similar_items, suggested_actions	Daily batch; on-demand for active sessions	Session start + nightly rebuild
PersonalizationEngine	personalized_rank, relevance_to_active_persona, feed_visibility_score	Session-scoped; per-user cache invalidated on persona update	Persona change; session start
MatchEngine	match_score (opportunity queries), talent_match_score (resource rows), investor_match_score (campaign rows)	On-demand via gRPC; cached for 5 min	New opportunity posted; user profile update
GraphEngine	dependency_depth, critical_path_flag, network_centrality, cluster_membership	Async after edge add/remove	GraphEdge mutation events
RiskEngine	risk_score, risk_breakdown, mitigation_gap_flags, at_risk_deadline_count	Plugin writeback: on_component_updated()	EventLog events: status change, budget overrun, due date approach
OptimizationEngine	resource_optimization_hints, budget_reallocation_suggestions,	Daily batch; on-demand for active projects	Sprint completion; milestone approach

Engine	Columns Contributed	Writeback Protocol	Update Trigger
	schedule_optimization_hints, predicted_completion_date		
SearchEngine	search_rank, full_text_index_status, keyword_suggestions	Plugin hook: on_component_created() + on_component_updated()	Component create/update
AllocationEngine	capacity_conflicts, allocation_recommendations, utilization_heatmap	Async after allocation change	ResourceAllocation on create/update events
IncentiveEngine	kogipoints_earned, incentive_streak, contribution_badges	Event-driven: on game event completion	Transaction completion; badge criteria met
BenefitsEngine	benefits_health_score, coverage_gap_flags, tax_savings_estimate, contribution_forecast	Weekly batch + on income event	New gig/contract income; quarterly review
GrantEngine	grant_match_score, eligibility_flags, recommended_grants	Weekly batch	Portfolio composition change; new grants in registry
CrowdfundingEngine	campaign_health_score, success_probability, investor_match_quality	Real-time for active campaigns; daily for inactive	Campaign backing event; velocity check
CollaborationEngine	collaboration_score, contributor_diversity_index, merge_conflict_count	After contribution events; daily rollup	Contribution accepted/rejected; merge conflict resolved
BookingEngine	availability_conflicts, booking_revenue_forecast, utilization_calendar	On booking event; daily calendar rebuild	New booking; cancellation; availability change
CRMEngine	lead_score, pipeline_value, follow_up_due_flags	Daily batch; real-time on inquiry	New inquiry; stage transition; activity logging
LogisticsEngine	logistics_status, itinerary_health, resource_routing_conflicts	On logistics event	Booking confirmed; equipment assigned; route changed
LinkNetworkEngine (KLNK)	link_count, inbound_link_count,	On LinkEdge create/delete	KLNK graph mutation events

Engine	Columns Contributed	Writeback Protocol	Update Trigger
	network centrality, link depth, linked_portfolio_ids		
IncomeProjection model	income_projection, earnings_trend, platform_diversification_score	Weekly batch	New gig/contract completion; quarterly earnings milestone

11.2 Plugin Writeback Protocol

The plugin writeback protocol is the mechanism by which kogi-engine sub-systems write computed values back into portfolio rows without bypassing the PermissionTier checks or EventLog requirements. All engine writebacks are treated as first-class EventLog mutations.

Protocol Step	Description
1. Engine computes value	The kogi-engine sub-system computes a new value for one or more AIColumns on one or more PortfolioComponents. This may be the result of a batch computation, a real-time signal, or an on-demand query.
2. Engine calls WritebackService	The engine calls the WritebackService gRPC endpoint with: component_id, column_id, new_value, confidence (0-1), source_engine, computation_timestamp.
3. Permission check	WritebackService validates that the writing engine has the AIWriter permission for the target column. This permission is granted at platform level and cannot be overridden by users.
4. PolicyEngine evaluation	WritebackService evaluates the PolicyEngine for any policies that might block or modify the writeback (e.g. a policy that requires human review before risk_score changes above a threshold).
5. EventLog append	A PortfolioEventKind::AIColumnUpdated event is appended to the component's EventLog with the full writeback details including source engine and computation_timestamp.
6. CellStore update	The new column value is written to the CellStore cache with a TTL determined by the column's update_frequency setting.
7. WebSocket broadcast	All clients currently viewing the affected sheet receive a real-time CellUpdateEvent for the changed columns.
8. KNTF notification	If the writeback changes a column value that crosses an alert threshold (e.g. health_score drops below 60, risk_score rises above 80), KNTF dispatches the appropriate notification.

12. Persistence & Storage Architecture

12.1 Storage Layer Design

Store	Technology	What Is Stored	Access Pattern
Primary Component Store	PostgreSQL + JSONB	PortfolioRow canonical records — ComponentMetadata + ComponentData serialized as JSONB with typed indexed columns for hot query fields (status, owner, type, created_at, health_score, visibility)	Read-heavy; write on mutation; partitioned by owner_entity_id
Graph Store	PostgreSQL (adjacency table)	GraphEdge table: source_id · target_id · edge_type · metadata JSONB · created_at. Indexed on source, target, and type. Includes InterPortfolioLink edges for KLNK.	Read: O(1) by source or target; Write: on edge create/delete
EventLog Store	PostgreSQL (append-only, partitioned by created_at)	Immutable event records: component_id · actor_id · identity_tag · event_type · delta JSONB · vector_clock · timestamp. Partitioned by month.	Append-only writes; range reads for audit; never updated or deleted
CRDT Buffer	Redis (hash by component_id)	CrdtLog operations awaiting apply; vector clock cache; hot-path LWW cell cache for sub-millisecond concurrent writes	High-frequency read/write; TTL-based expiry; flushed to PostgreSQL periodically
Cell Cache	Redis (hash by component_id:column_id)	Computed column values cached with TTL. Prevents re-computation on every read for expensive AI columns.	Read: O(1) by key; Write: on engine writeback or cell mutation; invalidated by TTL or explicit event
Link Network Store	PostgreSQL (LinkEdge table) + Redis (hot path)	InterPortfolioLink edges, ShadowRow metadata, MirrorColumn configurations, LinkForest snapshots	Read: graph traversal queries; Write: on link create/delete; forest snapshots materialized hourly
Identity & Profile Store	PostgreSQL (ProfilePartition table)	ProfilePartition records, VisibilityMask configurations, account-to-partition mappings, identity anchors	Read on every authenticated request; Write on

Store	Technology	What Is Stored	Access Pattern
			identity management actions
Search Index	Meilisearch / Typesense	Full-text index over component name, tags, bio, description, custom fields, identity handles, linked portfolio names	Read: search queries; Write: on component create/update via plugin hook; rebuilt daily
Snapshot Store	S3-compatible object store	Full PortfolioSystem snapshots serialized as compressed JSON. Keyed by snapshot_id; lifecycle-managed with configurable retention.	Write: scheduled + manual; Read: on restore; lifecycle: 90-day default retention
Analytics Store	ClickHouse (columnar)	High-throughput append of analytics events: views, clicks, engagements, shares, link traversals. Queried for aggregation by AnalyticsEngine.	Append-only writes; aggregate reads; never updated after write
Engine Feature Store	Redis + PostgreSQL (materialized views)	Pre-computed engine signals (health_score, risk_score, match_score, link_strength) stored as materialized views, refreshed on schedule and on trigger	Read: per-component on sheet load; Write: engine writeback via WritebackService
Shadow Row Store	PostgreSQL (shadow_rows table)	ShadowRow records with source_component_id, visible_columns, sync status, last_synced timestamps. Cross-user; indexed by host entity and source entity.	Read: on sheet load for host user; Write: on link creation + sync events

12.2 Indexing Strategy

Index	Type	Columns Indexed	Purpose
Primary Key Index	B-Tree (PostgreSQL)	component_id	O(1) component lookup
Owner Index	B-Tree	owner_entity_id + status	Fast "all my components" queries
Type Index	B-Tree	item_type + status + visibility	Sheet filter queries by type

Index	Type	Columns Indexed	Purpose
Status Index	B-Tree	status + updated_at	Recent active components; lifecycle queries
Visibility Index	B-Tree	visibility + item_type	Public component discovery
Financial Indexes	B-Tree	budget_utilization_pct + status; payment_status + due_date	Finance sheet queries; overdue invoice detection
Score Indexes	B-Tree	health_score DESC; risk_score DESC	Analytics sheet queries; health/risk rankings
Timestamp Indexes	B-Tree	created_at + updated_at + archived_at	Timeline queries; recent updates feed
Tag Index	GIN (PostgreSQL)	tags, hashtags, labels	Tag-based filtering and discovery
Full-Text Index	Meilisearch	name, description, custom fields, identity_handle	Universal search across all components
Hierarchy Edge Index	B-Tree (GraphEdge table)	source_id + edge_type; target_id + edge_type	O(1) parent/child lookups; tree traversal
KLNK Link Index	B-Tree (GraphEdge table)	edge_type=InterPortfolioLink; from_entity + to_entity	Link network traversal; mutual connection lookup
Identity Tag Index	GIN	identity_tags	Partition-scoped queries; profile view generation
ShadowRow Index	B-Tree	host_entity_id + source_component_id	Shadow row lookup by host + source
EventLog Partition Index	B-Tree (partitioned by month)	component_id + created_at	Audit trail queries; event replay

12.3 Snapshotting & Recovery

Operation	Trigger	Storage	Recovery
Auto Snapshot	Daily at 02:00 UTC per active portfolio	S3: {entity_id}/{date}.snap.json.gz	restore_snapshot() replays all ComponentData; post-restore EventLog preserved
Manual Snapshot	User or Oba triggers via UI or API	S3: {entity_id}/{user_id}/{timestamp}.snap.json.gz	Same as auto snapshot

Operation	Trigger	Storage	Recovery
Checkpoint	Before any bulk operation (bulk status change, import, federation sync, identity merge)	Redis (temporary) + S3 (permanent after confirmation)	Checkpoint restore is instant for recent (Redis) or slower for S3
Identity Merge Checkpoint	Before any CrossIdentityMerge operation	S3 with merge_checkpoint tag	Special restore path that reverses identity tag assignments
Link Network Snapshot	Hourly LinkForest snapshot per entity	PostgreSQL materialized view + S3 daily backup	Used for link graph analytics and restore of disconnected ShadowRows
Federation Sync Checkpoint	Before each CRDT sync with a federation peer	CrdtLog buffer in Redis	Replay buffered operations from last successful sync point on reconnect

13. Security, Access Control & Compliance

13.1 Permission Tier Hierarchy

Tier	Ordinal	Granted Capabilities	Sheet Access
Public (Unauthenticated)	0	Read-only access to rows where visibility = Public. No mutations permitted.	SHT-024 (Feed), SHT-026 (Marketplace) — public rows only
Viewer	1	Read-only access to all rows the caller is permitted to see. No mutations.	All sheets — permitted rows only; computed columns visible
Subscriber	2	Read access + bookmark, save, follow, subscribe actions. No content mutations.	All permitted sheets + SHT-025 (Contacts) subscriber view
Contributor	3	Comment, react, poll, join, submit contributions. Cannot edit parent resource directly.	All permitted sheets + contribute actions on SHT-020/SHT-021
Editor	4	Edit content, bump version, create child items, modify columns. Cannot delete or transfer ownership.	All sheets with edit capability; can add custom columns
Manager	5	Manage settings and members, configure views, govern access, configure board modes.	All sheets + sheet management; can publish views as templates
Owner	6	Full ownership privileges: all actions, all columns, all sheets, delete, transfer, identity configuration.	All sheets including SHT-029 (Event Log), SHT-030 (Snapshots), SHT-032 (Identity)
Admin	7	Super-admin / platform-level override. Can access any component subject to legal holds.	All sheets; audit access to all EventLogs; compliance reporting

13.2 Row-Level Security (RLS)

Every query against the PortfolioRow store is automatically scoped by PostgreSQL Row-Level Security policies. The RLS layer evaluates the caller's identity and permission tier against the component's visibility field and ACL before returning any data.

Visibility Level	Who Can Read	Who Can Write	KLNK Link Visibility
Private	Owner + explicitly invited users with Viewer+ tier	Owner + Editors with explicit grant	Not discoverable via KLNK. Links still possible if directly initiated by owner.

Visibility Level	Who Can Read	Who Can Write	KLNK Link Visibility
Protected	Owner + approved followers + explicit invitees	Owner + Editors	Discoverable to approved followers; links shown to connections
Internal	All members of the owning organization	Owner + Editors + org Managers	Visible to org members in KLNK org tree; not in public graph
Public	Anyone including unauthenticated visitors	Owner + Editors	Fully discoverable in KLNK; appears in public graph traversals
Unlisted	Anyone with the direct link/URL; not indexed	Owner + Editors	Not indexed in KLNK; direct links still work
DraftOnly	Owner only (even co-editors cannot see during draft)	Owner only	Not discoverable; no KLNK visibility
Custom (ACL)	Users matching explicit ACL rules defined by owner	Users with Editor role in ACL	Discoverable to ACL members; custom LinkVisibility per edge type

13.3 Column-Level Visibility

Individual columns can have their own visibility level, independent of the row's visibility. This is particularly important for sensitive columns (financial details, contact info, health data) that should be visible to the row owner but not to followers or the public.

Column Sensitivity Tier	Default Columns	Visible To
Public	name, slug, status, tags, description, views, followers, component_type, item_type	Everyone who can see the row (based on row visibility)
Follower	progress_pct, health_score, created_at, updated_at, lifecycle_stage, skill tags	Followers and above
Connection	owners (names), budget_allocated (if public project), milestone dates	Trusted connections and above
Contributor	detailed analytics, team member list, detailed timeline, work items	Contributors and above
Editor	budget_spent, financial details, invoice_refs, risk_flags, compliance_flags	Editors and above
Owner-only	bank_account_ref, tax_category, personal financial details, benefit balances, identity partition config, VisibilityMask rules	Owner only (even when row is Public)

Column Sensitivity Tier	Default Columns	Visible To
AI-Internal	raw engine signal inputs, algorithm parameters, model confidence intervals	System only; never exposed to user queries

13.4 Multi-Identity Privacy & Compliance

Privacy Feature	Description
Identity Isolation	Rows tagged exclusively to an Isolated identity are never accessible through other identity contexts, even when authenticated as the same Sovereign Entity. Full hardware-boundary equivalent at the data layer.
Cross-Identity Audit Trail	Any access to cross-partition data by a secondary identity is logged as a CrossPartitionAccessEvent in the immutable EventLog. This provides a full audit trail for identity separation.
PII Classification	Legal name, phone numbers, private emails, financial account details, health data, and government IDs are classified as PII. Stored with field-level encryption (AES-256). Never returned in bulk exports without explicit consent per field.
GDPR Right to Erasure	Supports GDPR Article 17: user can request erasure of personal data from all sheets and KLNK link records. EventLog entries are anonymized (actor_id replaced with anonymous_actor_id). Cryptographic audit chain maintained without PII.
CCPA Data Portability	Full portfolio data export in JSON-LD format including all sheets, event logs, analytics, contributions, and financial summaries. Delivered as encrypted, signed archive.
Identity Pseudonymity	Any identity can be configured as pseudonymous: the @handle is not linked to legal identity in any publicly accessible record. Sovereign Entity mapping stored only in encrypted KPID vault.
Data Minimization	Only data required for the user's configured use cases is collected. Unused columns are not instantiated. Engine signals are only computed for columns the user has enabled in their sheet configuration.
Consent Management	Granular consent controls per data type, engine integration, and KLNK link type. Users can revoke consent for specific data uses at any time. Revocation propagates to linked engine subscriptions.

14. API Surface (KPMS · REST + gRPC)

14.1 Portfolio REST API — Core Endpoints

Endpoint	Method	Description	Auth Required
/portfolio/:id	GET	Read a single PortfolioComponent by ID (visibility-gated by caller's tier)	Viewer+
/portfolio/@:handle/:slug	GET	Read component by identity handle and slug	Viewer+
/portfolio	POST	Create a new component (item or container)	Contributor+
/portfolio/:id	PATCH	Update component fields (owner/editor only)	Editor+
/portfolio/:id	DELETE	Soft-archive component	Owner
/portfolio/:id/children	GET	List direct children in hierarchy	Viewer+
/portfolio/:id/descendants	GET	List all transitive descendants with pagination	Viewer+
/portfolio/:id/attach/:child_id	POST	Attach a child component (creates Hierarchy edge)	Editor+
/portfolio/:id/attach/:child_id	DELETE	Detach a child component	Editor+
/portfolio/:id/actions	POST	Perform an ActionKind: like, share, follow, invest, donate, post, tag	Contributor+
/portfolio/:id/analytics	GET	Get computed analytics for a component	Viewer+
/portfolio/:id/health	GET	Get PortfolioHealth model output with dimension breakdown	Viewer+
/portfolio/:id/eventlog	GET	Get EventLog for a component (paginated)	Editor+
/portfolio/:id/allocations	GET	Get resource allocations for a component	Editor+
/portfolio/:id/allocate	POST	Create a resource or budget allocation	Manager+
/portfolio/:id/record-spend	POST	Record budget spend against a component	Editor+
/portfolio/search	GET	Full-text + faceted search across the registry	Viewer+
/portfolio/query	GET/POST	PQL (Portfolio Query Language) query endpoint	Editor+
/portfolio/:id/contributions	GET	List ContributionRecords for a shared component	Contributor+
/portfolio/:id/contribute	POST	Submit a contribution to a shared component	Contributor+
/portfolio/:id/snapshot	POST	Create a named snapshot	Owner
/portfolio/restore/:snapshot_id	POST	Restore from a snapshot (requires confirmation)	Owner

Endpoint	Method	Description	Auth Required
/portfolio/export/:sheet_id	GET	Export sheet as CSV, XLSX, or JSON (content-type negotiation)	Editor+
/portfolio/:id/crdt/sync	POST	Submit a CrdtLog batch for federation synchronization	System (federation peers only)

14.2 Sheet & View API

Endpoint	Method	Description
/sheets	GET	List all sheets accessible to the authenticated user (with row counts and last_updated)
/sheets/:sheet_id	GET	Get sheet definition and metadata
/sheets/:sheet_id/rows	GET	Query rows on a sheet with filter, sort, group, pagination (PQL-compatible params)
/sheets	POST	Create a custom sheet (SHT-034 type)
/sheets/:sheet_id	PATCH	Update sheet definition (column schema, default sort/group)
/views	GET	List saved views for the authenticated user
/views	POST	Create a new saved view
/views/:view_id	PATCH	Update a saved view (filters, sorts, groups, highlight rules)
/views/:view_id/rows	GET	Render rows for a specific saved view (applies all view filters/sorts/groups)
/views/:view_id/share	POST	Share a view with specific users or make it public template
/columns	GET	List column schema for a sheet (including custom columns)
/columns	POST	Add a custom column to a sheet
/columns/:col_id	PATCH	Update a custom column definition
/columns/:col_id	DELETE	Remove a custom column (with data preservation option)
/columns/:col_id/compute	GET	Get computed value for a specific ComputedColumn on a row

14.3 Link Network API

Endpoint	Method	Description
/klnk/nodes/:id/tree	GET	Get LinkTree rooted at node_id with depth, edge_type filter, and direction params
/klnk/nodes/:id/forest	GET	Get complete LinkForest for the entity owning this node
/klnk/nodes/:id/ego	GET	Get ego network (depth 1+2) centered on node_id
/klnk/edges	POST	Create a new LinkEdge (link request). Triggers consent flow for consent-required edge types.
/klnk/edges/:edge_id	DELETE	Remove a LinkEdge. Triggers ShadowRow disconnection protocol.
/klnk/shadow-rows	GET	List all ShadowRows in the caller's spreadsheet
/klnk/shadow-rows/:id/sync	POST	Manually trigger a MirrorColumn sync from source
/klnk/shadow-rows/:id/columns	GET/PATCH	Get or update the visible_columns and write_back_columns configuration for a ShadowRow
/klnk/discover	GET	Discover link targets matching a query, ranked by match_score and link_type relevance
/klnk/graph/path	GET	Find shortest path between two nodes (Dijkstra with edge weight)
/klnk/graph/clusters	GET	Identify community clusters (Louvain). Useful for ecosystem analysis.
/klnk/graph/centrality	GET	Compute betweenness, closeness, and eigenvector centrality for a node set
/klnk/subscriptions	GET/POST/DELETE	Manage link change subscriptions: webhooks and Kafka events for linked node changes
/klnk/sheet	GET	Render the Link Network as a spreadsheet sheet (SHT-031 view)

14.4 Identity & Profile API (KPID)

Endpoint	Method	Description
/identity/partitions	GET	List all ProfilePartitions for the authenticated Sovereign Entity
/identity/partitions	POST	Create a new ProfilePartition (new identity/profile)

Endpoint	Method	Description
/identity/partitions/:id	GET/ PATCH	Read or update a ProfilePartition configuration
/identity/partitions/:id/visibility-mask	GET/ PUT	Get or replace the VisibilityMask for a partition
/identity/partitions/:id/rows	GET	List all rows tagged to this partition
/identity/rows/:component_id/tags	GET/ POST /DELETE	Get, add, or remove identity_tags on a specific row
/identity/accounts	GET	List all accounts associated with the Sovereign Entity
/identity/accounts/:id/scope	GET/ PUT	Get or update the access scope for a specific account
/identity/merge	POST	Initiate a CrossIdentityMerge operation (requires checkpoint creation + confirmation)
/identity/handoff	POST	Transfer an identity partition to a different Sovereign Entity
/identity/:handle/public-view	GET	Get the public spreadsheet view for a specific @handle (respects VisibilityMask)
/identity/split-policy	GET/ PUT	Get or update the SplitPolicy governing cross-partition data access

14.5 Portfolio Query Language (PQL)

PQL is a structured query interface for the portfolio registry — SQL-inspired but typed to the PortfolioRow schema. Exposed via the /portfolio/query endpoint and the QueryService gRPC method. Supports cross-partition queries for authenticated owners and filtered cross-portfolio queries for link network traversals.

PQL Feature	Description	Example
SELECT	Choose which columns to return. Use * for all visible columns. Computed columns resolved at query time.	SELECT id, name, health_score, budget_remaining
FROM rows	Always queries the PortfolioRow store. No FROM variants — single table model.	FROM rows
WHERE	Filter predicates: all ViewFilter types supported. AND/OR nesting. Subquery conditions.	WHERE item_type = "Project" AND health_score < 70

PQL Feature	Description	Example
ORDER BY	Multi-column sort. ASC/DESC. NULLS FIRST/LAST.	ORDER BY due_date ASC NULLS LAST, health_score DESC
GROUP BY	Aggregate with SUM/AVG/COUNT/MIN/MAX. Supports rollup expressions.	GROUP BY program_ref.name, ROLLUP(quarter)
LIMIT / OFFSET	Pagination. Max LIMIT 10000 per query for non-admin callers.	LIMIT 50 OFFSET 100
JOIN	Cross-relation join. Join linked components via RelationField columns.	JOIN program_ref ON program_ref.id = program_id
IDENTITY SCOPE	Filter to rows tagged to a specific identity partition.	IDENTITY "@jordan-dev"
SHADOW INCLUDE	Include ShadowRows from linked portfolios in results.	INCLUDE SHADOW ROWS FROM "@partner-org"
AS_OF	Time-travel query: returns row state at a specific timestamp using EventLog replay.	AS_OF "2026-01-01T00:00:00Z"

15. Export, Integration & Embedding (KPEX)

15.1 Export Formats

Format	API Method	Notes	Identity Scope
CSV	GET /portfolio/export/:sheet_id?format=csv	Standard delimiter-separated export of visible columns. Computed columns included as static values at export time.	Applies VisibilityMask of exporting identity
XLSX	GET /portfolio/export/:sheet_id?format=xlsx	Multi-sheet Excel workbook: main sheet + summary pivot + chart data. Uses openpyxl pipeline.	Same as CSV
JSON	GET /portfolio/export/:sheet_id?format=json	Raw PortfolioRow JSON array — full schema including all fields, relationships, and computed values.	Owner-level: all columns. Others: visible columns only.
JSON-LD	GET /portfolio/export/:sheet_id?format=jsonld	Linked Data export with schema.org and kogi-vocab type annotations — portable, semantic, machine-readable.	Public-facing only; applies Public visibility mask
Markdown	GET /portfolio/export/:sheet_id?format=md	Markdown document: section headings per group, tables per component type. Suitable for portfolio showcases and README generation.	Configurable — applies selected identity's public mask
PDF Report	GET /portfolio/export/:sheet_id?format=pdf	Rendered portfolio report document — styled with kogi brand template, charts, health summaries, key metrics.	Configurable per export
vCard Bundle	GET /portfolio/export/contacts?format=vcf	Exports ContactBook rows as a vCard (.vcf) bundle for import into external contact managers.	Owner only
GDPR Archive	POST /portfolio/export/gdpr-archive	Complete data export: all sheets, all partitions, full EventLog, analytics, contributions, financial summaries. Encrypted + signed.	Owner only; triggers audit log entry
Webhook Push	POST /portfolio/webhooks	Real-time push of EventLog events or row change deltas to a registered external URL on every mutation.	Configurable per webhook; partition-scope d
Embed Widget	GET /portfolio/embed/:view_id	Embeddable iframe/script rendering a read-only view of a public sheet or portfolio in an external webpage.	Public rows only; respects public VisibilityMask

Format	API Method	Notes	Identity Scope
Developer API	REST + gRPC	Full programmatic access via kogi-dev SDK. Supports streaming for large result sets.	Per API key permissions

15.2 Import Sources

Import Source	Method	Column Mapping	Identity Assignment
CSV / XLSX Upload	File upload → Oba-assisted column mapping wizard → batch create	Intelligent type inference + user confirmation	Prompt user to select which identity partition to assign imported rows
Google Sheets	OAuth + Sheets API → pull → import wizard	Schema mapping with type coercion	Same as CSV
Notion	Notion integration (API key) → database pull → property mapping	Notion property types mapped to PortfolioColumn types	User selects partition
Airtable	Airtable API → base pull → field mapping	Airtable field types → PortfolioColumn types	User selects partition
Jira	Jira API → issue pull → story type mapping	Issue types → StoryType enum; fields → custom columns	Defaults to primary work identity
Asana	Asana API → project pull → task mapping	Project → Program/Project rows; tasks → Task rows	User selects partition
GitHub / GitLab Issues	Issues API → pull → type classification	Issue labels → story_type; milestones → milestones column	Defaults to developer identity if present
Quickbooks / Wave	Accounting API → transaction pull → financial mapping	Transactions → Finances sheet rows; categories → tax_category	Defaults to financial identity
Gig Platform Earnings	Platform API (DoorDash, Uber, etc.) or CSV upload	Earnings fields → Gig row columns; tips → tips column	Defaults to gig worker identity partition
ContactBook Import	vCard, Google Contacts, Outlook CSV	Contact fields → Profile row columns	User selects partition; defaults to primary professional identity

16. Portfolio Template System

16.1 Template Types

Template	Pre-configured For	Included Sheets & Views	Identity Config
Independent Worker Starter	Solo freelancer / gig worker; first portfolio setup	Master Registry, Work & Gigs, Finances, Portable Benefits, Timeline, Contacts, Link Network	Single primary identity; default professional profile
Multi-Identity Freelancer	Worker with multiple clients/roles requiring separate public profiles	All starter sheets + Identity & Profiles sheet; pre-configured with 2 ProfilePartitions	2 identities: professional + personal/creative
Software Developer Portfolio	Developer with projects, open-source contributions, and client contracts	Projects (Agile board), Tasks & Backlog, Assets (repos), Artifacts (releases), Finances, Contacts, Link Network	Dev identity with GitHub federation link
Creative Portfolio	Designer, artist, musician, content creator	Projects, Artifacts (creative works), Marketplace Listings, Finances, Community Feed, Contacts, Link Network	Creative identity + pseudonymous public identity option
Freelance Consultant	Consultant with multiple client engagements	Work & Gigs (contract-focused), CRM view, Finances, Timeline, Contracts & Deliverables, Contacts	Professional identity; client-facing profile partition
Cooperative / Collective	Multi-member cooperative or collective	Shared Portfolios, Group Economics, Collaboration, Governance, Finances (cooperative treasury), Organizations, Link Network	Org identity as primary; member identities linked
Startup Founder	Early-stage startup with team, product, investors	Projects, Team (shared), Equity & Crowdfunding, Grants, Roadmap, Finances, Contacts, Risk Register, Link Network	Founder identity + company org identity
Gig Worker Benefits Optimizer	Gig worker focused on portable benefits and income optimization	Work & Gigs, Portable Benefits, Finances, Grants & Microfinancing, Income Projection view	Single gig worker identity; benefits-focused profile
Event Producer / Booking	Event or touring professional with bookings, crew, logistics	Work & Gigs (booking), Timeline (tour calendar), Resources (crew, equipment), Finances, Logistics	Performance identity + production identity
Organization / DAO	Formal organization with governance,	Organizations, Shared Portfolios, Group Economics, Governance, Finances	Org entity as root; member user identities

Template	Pre-configured For	Included Sheets & Views	Identity Config
	treasury, and member management	(treasury), Member Directory, Link Network	linked via OrgMembership edges
Research & Academic	Researcher, academic, or knowledge worker	Projects (research), Artifacts (papers, datasets), Grants, Timeline, Contacts, Knowledge Registry	Academic identity with institution federation link
Multi-Platform Gig Aggregator	Worker on 5+ gig platforms who needs unified income and benefits tracking	Work & Gigs, Portable Benefits (deep dive), Finances, Income Projection, Platform Dashboard view	One identity per platform with unified root spreadsheet; cross-platform earnings aggregation

17. Oba AI Integration — Spreadsheet Intelligence

Oba is the AI chief-of-staff that lives inside the portfolio spreadsheet. It reads every row, every column, and every engine signal — and surfaces proactive, context-aware intelligence directly in the spreadsheet UI. Oba does not just answer questions; it notices what the user should notice, flags what they should act on, and prepares actions for their approval.

17.1 Oba Capabilities in the Spreadsheet

Capability	Description	Trigger
Row Annotations	Oba attaches inline annotations to rows: "This project is 14 days behind schedule — adjust due date or reduce scope?" · "This grant deadline is in 5 days — draft application now?"	Health/risk threshold crossed; deadline approach
Column Completion	Oba suggests values for empty required fields: "Due date is missing — suggest April 30 based on similar projects?" · "No risk flags — want me to analyze and create an initial risk assessment?"	Required field empty; first open after row creation
Smart Filters	"Show me everything at risk this week" → Oba translates natural language to ViewFilter: { risk_score > 70 OR due_date < TODAY+7 AND status = Active }	Natural language query in search bar
Formula Suggestions	When a user adds a custom column, Oba suggests formulas: "You're tracking revenue and expenses — want me to add a profit margin % formula column?"	New column creation of NumberField or CurrencyField type
Batch Actions	"Archive all completed projects older than 90 days" → Oba prepares a bulk archive action for user confirmation before executing	Natural language instruction
Sheet Generation	"Create a sheet showing all my income sources this quarter grouped by platform" → Oba generates a ViewDefinition and creates the custom sheet	Natural language request for new view
Daily Health Briefing	Oba portfolio briefing: top health risks, upcoming deadlines, pending approvals, contribution opportunities, suggested grant matches, network growth alerts	Scheduled: daily morning; on-demand via Oba chat
Anomaly Alerts	Oba proactively flags: budget overrun approaching (>80% utilization), velocity drop in sprint, stalled project (no	AnomalyEngine detection; threshold

Capability	Description	Trigger
	updates 14 days), coverage gap in benefits, unusual link network pattern	crossing; inactivity timer
Import Mapping	During CSV/XLSX import, Oba suggests column mappings: "This looks like 'Project Name' — map to PortfolioRow.name?"	Import wizard launch
Link Network Suggestions	"Your portfolio is highly visible to investors based on link patterns — consider publishing your Equity & Crowdfunding sheet" · "Alex (3 mutual connections) has skills that match your current project gap"	KLNK link analysis; MatchEngine collaborator suggestion
Identity Optimization	"Your @jordan-dev identity hasn't been updated in 30 days — want me to update your public showcase with your recent project completions?"	Profile staleness detection; new completed work not reflected in public view
Contribution Matching	For crowdresourced components: "Alex has the skills listed in this project's requirements — send collaboration invite?"	MatchEngine talent match for open contribution requirements
Narrative Generation	"Write a portfolio highlight for my top 3 projects this quarter" → Oba generates structured portfolio narrative from row data for resumes, proposals, grant applications	User request; quarterly review reminder
Grant Application Drafting	"I found 3 grants you may be eligible for — want me to draft the eligibility checklist and start the application for the best match?"	GrantEngine match score crosses threshold; new grant in registry matches portfolio
ShadowRow Health	"The ShadowRow for 'Design Collective Project' hasn't synced in 72 hours — the source may have changed. Review or refresh?"	ShadowRow sync failure; stale threshold crossed
PQL Query Builder	Oba translates natural language into PQL queries: "Show me all projects over budget sorted by overrun amount" → full PQL statement	User question in Oba chat that implies a data query

17.2 Oba Identity & Network Workflows

Oba has specific capabilities tailored to the multi-identity and link network features that are unique to the KMDSS:

Workflow	Description
Identity Audit	Oba reviews all ProfilePartitions and identifies: rows that should be visible on a profile but aren't tagged, rows that are public but contain sensitive data that should be owner-only, profiles that are incomplete or stale. Generates an action list for owner review.
Visibility Review	"Before you publish this project as Public, here are 3 columns that contain owner-only data that will become visible to followers: bank_account_ref, tax_category, and expense_details. Confirm or adjust visibility."
Link Network Introduction	"You and @partner-org have 8 mutual connections and both work in fintech. You've referenced their playbook twice. Would you like me to draft a collaboration inquiry?"
ShadowRow Curator	Oba monitors all ShadowRows in the user's spreadsheet. Flags rows that are disconnected, stale, or where the source component's visibility changed. Prepares update or archive actions.
Cross-Portfolio Due Diligence	When a user is considering a deal with a linked portfolio: "Here's a summary of @contractor-x's linked portfolio: 12 completed projects (avg health score 84), 3 open projects, response time 1.2 days, 47 network connections including 4 people you know."
Federation Health Monitor	For federated portfolios: monitors CRDT sync health across all federation nodes. Alerts on divergence, identifies conflict hot spots, suggests conflict resolution strategies.
Income Attribution by Identity	"Your @jordan-dev identity generated \$34,200 this quarter; @studio-collective generated \$18,500. Both are under-reporting portable benefits contributions — want me to create the missing BenefitAccount rows?"

18. Platform Integration Map

kogi-* Module	Dependency on KMDSS	Data Flow	Identity Scope
kogi-home	Reads Master Registry for dashboard overview: active program count, portfolio health summary, wallet balance, upcoming deadlines	Read: portfolio → home dashboard; respects active identity context	Shows data for currently active identity
kogi-office (boards/timelines)	Reads SHT-004 (Projects) + SHT-005 (Tasks) for board and gantt rendering; writes task status transitions and sprint completions back	Read + Write: portfolio ↔ office; real-time via CRDT	Work identity partition by default; configurable
kogi-bank	Reads SHT-009 (Finances), SHT-015 (Benefits), SHT-016 (Grants), SHT-017 (Crowdfunding); writes payment events, benefit balance updates, and ledger transactions back	Read + Write: portfolio ↔ bank; financial columns always owner-only visibility	All financial data always scoped to Sovereign Entity; never identity-partitioned below owner level
kogi-marketplace	Reads SHT-026 (Marketplace Listings) for published assets; writes investment/donation action events; reads talent from SHT-006 (Resources)	Read + Write: portfolio ↔ marketplace; KLNK link created on deal acceptance	Public identity used for marketplace presence; deal rows linked to work identity
kogi-community	Reads public rows from Master Registry for community feed; writes engagement events (likes, shares, mentions) to analytics columns	Read + Write: portfolio ↔ community; engagement events update AnalyticsColumn values	Community presence uses the active public identity (@handle shown on posts)
kogi-exchange	Reads SHT-017 (Exchange), SHT-027 (Deals); writes deal execution events and matched allocation records	Read + Write: portfolio ↔ exchange; InvestedIn KLNK edges created on investment	Investment identity or primary financial identity
kogi-profile (KPRF)	Profile rows ARE PortfolioComponents in SHT-025; linktree leaves stored as Resource rows; linked accounts sync to asset columns	Read + Write: portfolio ↔ profile; profile update regenerates public view	Each ProfilePartition generates one public profile at its @handle URL
kogi-engine	Reads all sheets via EventLog subscription for feature extraction; writes computed column values back via WritebackService	Read + Write: portfolio ↔ engine; all engine signals are AIColumns	Engine operates on full Sovereign Entity scope; per-column visibility mask applied before

kogi-* Module	Dependency on KMDSS	Data Flow	Identity Scope
			any public exposure
kogi-studio	Reads and writes Artifact rows (creative outputs); links design assets to parent Project rows via Hierarchy edges	Read + Write: portfolio ↔ studio; creative outputs create new Artifact rows	Creative identity partition; artifacts tagged to creative profile
kogi-dev (API/SDK)	Full read/write access to all sheets and rows via kogi-dev API; PortfolioPlugin trait for custom computed columns and automation	Full bidirectional: portfolio ↔ developer; OAuth-scoped per API key	API key scoped to specific partitions and sheets at creation
KOGI-MANAGER	Reads PolicyEngine attachments; writes RBAC policies and governance configurations; enforces PermissionTier on all mutations	Policy: KOGI-MANAGER → portfolio; policy updates trigger PolicyEngine re-evaluation	Platform-admin scope; all partitions
KOGI-APPSTORE	Distributes Portfolio Templates, custom column sets, computed model plugins, view theme packages	Distribution: APPSTORE → portfolio templates; template instantiation creates new sheet views	Template installation applies to selected identity partition

19. Core User Flows — End-to-End

19.1 Independent Worker — Setup Master Spreadsheet

14. Create account → Kogi onboarding wizard launches Portfolio Template Selector.
15. Choose template: "Independent Worker Starter" → root PortfolioSystem provisioned with 8 default sheets.
16. Identity setup: wizard prompts for @handle → primary ProfilePartition created.
17. Profile completion: Oba prompts for skills, availability, rate → Resource row created; BenefitAccount rows provisioned.
18. Platform connections: link gig platforms (DoorDash, Uber, etc.) → provider integrations activated; Gig rows auto-imported.
19. Bank connection: link kogi-bank → financial columns populated; tax settings configured.
20. Master spreadsheet is live: Master Registry shows first set of rows; Finances Sheet shows first income entries; Benefits Sheet shows placeholder BenefitAccounts.

19.2 Creating and Linking a Second Identity

21. Open SHT-032 (Identity & Profiles sheet) → "Add Identity" → enter @handle for new creative identity.
22. New ProfilePartition provisioned. VisibilityMask configured: creative projects Public; financial data Owner-only.
23. Tag existing rows: select Artifact rows for design work → "Tag to Identity" → "@jordan-art" applied.
24. New rows created in creative context: new Projects, Artifacts, Gig rows auto-tagged to active identity via session context.
25. Configure public view: select columns visible on @jordan-art public profile → save as public_sheet_view.
26. Link network: @jordan-art becomes a KLNK node. Others can follow, discover, and link to this identity independently from @jordan-dev.

19.3 Connecting Two Portfolios (KLNK Link)

27. User A (@jordan-dev) finds User B (@studio-collective) on the platform via Discover feed.
28. Jordan clicks "Connect" on a specific project in the studio-collective portfolio.
29. KLNK creates a pending Collaborates LinkEdge. @studio-collective receives a link request notification (KNTF).
30. @studio-collective reviews request, configures which columns of their project will be visible in Jordan's ShadowRow, accepts.
31. LinkEdge confirmed. ShadowRow created in Jordan's spreadsheet. Studio-collective's project appears in Jordan's SHT-020 (Shared Portfolios) sheet as a linked row with the configured columns visible.
32. Jordan can now see updates to studio-collective's project (within visibility config) in real-time via MirrorColumn sync.

33. Both users' link_count column updates. The edge appears in both users' SHT-031 (Link Network) sheets.

19.4 Cooperative Monthly Distribution

34. Month-end: cooperative treasury has surplus. Treasurer opens SHT-018 (Group Economics sheet).
35. Oba detects month-end trigger and pre-computes Shapley Value distribution based on contribution weights from SHT-021.
36. Oba presents distribution preview: each member row shows proposed distribution amount and attribution weight breakdown.
37. Treasurer initiates GovernanceProposal → members receive KNTF notification → vote via SHT-019 (Organizations sheet).
38. Quorum reached, proposal passes → Oba prepares batch payout transaction set in kogi-bank.
39. Multi-sig approvers confirm → batch executed → each member's Payroll Wallet credited.
40. Payroll Journal entries update cooperative's SHT-009 (Finances sheet). Attribution weights recomputed by CollaborationEngine.

19.5 Querying the Link Network Forest

41. User opens KLNK panel from any row with link_count > 0.
42. LinkTree query: GET
/klnk/nodes/{component_id}/tree?depth=3&edge_type=Collaborates,InvestedIn,Contracted
43. KLNK renders tree: root node = selected component; branches = linked components in connected portfolios; leaves = third-degree connections.
44. User can filter by edge type, visibility, mutual connection count, or engine score.
45. Clicking a node navigates to the ShadowRow for that component (if one exists) or shows the public view of the linked component.
46. User selects "Link Forest View" to see all trees rooted at all their identities side-by-side — the full picture of their network.
47. Export: GET /klnk/sheet renders the same data as SHT-031 — all link edges as spreadsheet rows for bulk analysis or export.

20. Open Items & Development Roadmap

20.1 Current Open Items (v3.0 → v3.5)

Item	Priority	Description
Persistence Layer Pluggability	P0	PortfolioSystem is currently in-memory with PostgreSQL being integrated. Full PortfolioStore trait abstraction needed for pluggable backends (SQLite for local, PostgreSQL for cloud, CouchDB for P2P).
Status CRDT Merge Strategy	P0	apply_crdt_op() currently skips status mutations. A custom CRDT lattice based on ComponentStatus lifecycle ordering must be specified before multi-node deployments.
Cross-Partition Write Policy	P0	SplitPolicy is defined but enforcement at the Substrate layer is not yet implemented. Cross-partition edits must be validated before any multi-identity deployment.
EventLog Flush to Data Lake	P1	10,000 event cap with FIFO eviction risks historical loss. Plugin hook must flush older events to ClickHouse / S3 before eviction, with full restoration path.
KLNK Consent Workflow	P1	For consent-required edge types (Collaborates, InvestedIn), the consent flow (invitation, negotiation, column visibility configuration) is specified but not yet implemented in the LinkEdge service.
ShadowRow Real-Time Sync	P1	MirrorColumn sync currently batch-based. Real-time sync via Kafka event subscription from source component's EventLog needs to be implemented and load-tested.
AI Agent Writeback Protocol	P1	WritebackService is defined but not yet enforcing the full permission check and audit trail. The typed protocol for engine writeback must be hardened before production.
Formula Engine (v1)	P1	ColumnComputer formula engine is defined but not fully implemented. v3.0 must ship the complete expression language evaluator with all listed functions including RELATED() and engine functions.
PQL Full Implementation	P2	PQL parser and evaluator must be implemented against the PostgreSQL storage layer with full JOIN support for RelationField columns and SHADOW INCLUDE syntax.
Real-Time Collaborative UI	P2	WebSocket-based live cursor indicator and cell update propagation — the full multi-user spreadsheet experience with presence indicators.
Link Forest Visualization	P2	The KLNK Forest and Tree rendering modes (force-directed graph, dependency tree) need a production-quality visualization component. D3.js + WebGL for large graphs.
Spreadsheet Import Pipeline	P2	CSV/XLSX import with Oba-assisted column mapping wizard and batch row creation with identity partition assignment and progress tracking.

Item	Priority	Description
Identity Merge Conflict Resolution	P2	CrossIdentityMerge operation needs a UI-level conflict resolution wizard for cases where both source and target identities have rows with the same component_id.
KLNK Centrality Computation	P3	Betweenness and eigenvector centrality for the full KLNK graph is computationally expensive at platform scale. Needs a distributed approximation algorithm (e.g. sampled random walk) for graphs >1M nodes.
Template Marketplace	P3	KOGI-APPSTORE integration for distributing, rating, and purchasing portfolio templates; template versioning and update notifications.
On-Chain Identity Anchoring	P3	For pseudonymous identities requiring external verifiability: DID (Decentralized Identifier) anchoring for identity handles via W3C DID spec. Enables cross-platform reputation portability.
Federated MatchEngine	P3	MatchEngine operating across ShangoOS federated portfolio nodes — cross-federation talent and resource matching against ShadowRows.

20.2 Version Roadmap

Version	Milestone	Key Deliverables
v2.2 (Current)	Portfolio Spreadsheet Foundation	Master Registry Sheet · Hierarchy Sheet · Projects Sheet · Finances Sheet · Benefits Sheet · Work & Gigs Sheet · PortfolioHealth v1 · Basic CRDT · PostgreSQL persistence · Single-identity mode
v2.5	Intelligence & Collaboration	AI ComputedColumns (health, risk, income projection) · CollaborationEngine · Shared Portfolios · Contribution Attribution · CrowdfundingCampaign · Grant & Crowdfunding Sheets · WritebackService v1
v3.0	Full Spreadsheet Platform	PQL · Formula Engine · All 35 Sheets · All Board Modes · Template Marketplace · Real-time Collaboration UI · Full Import/Export Pipeline · Oba Spreadsheet Intelligence · KLNK v1 (basic link creation and ShadowRows)
v3.5	Link Network & Multi-Identity	Full KLNK graph API · LinkForest and LinkTree rendering · Multi-identity KPID system · CrossIdentityMerge · ShadowRow real-time sync · Identity-aware VisibilityMask · KLNK centrality computation · Federation of portfolio nodes
v4.0	Autonomous Intelligence & Federation	Oba autonomous portfolio management (approval-first) · Predictive analytics and scenario simulation · AI-generated portfolio narratives · Cross-platform reputation portability via verifiable credentials · DID anchoring · Federated MatchEngine across ShangoOS

Version	Milestone	Key Deliverables
v4.5	Web3 & Decentralization	On-chain identity anchoring · Decentralized portfolio storage option (IPFS + CouchDB) · Smart contract-backed cooperative distribution · On-chain equity and cap table management · Cross-federation KLNK traversal across decentralized nodes

20.3 Glossary

Term	Definition
KMDSS	Kogi Master Distributed Spreadsheet System — the entire system described in this document
KPMS	Portfolio Management System — the master orchestration layer of the KMDSS
KPSS	Portfolio Spreadsheet Substrate — the low-level Rust data engine: PortfolioRow, ColumnSchema, CellStore
KPRG	Portfolio Component Registry — the canonical master index of all PortfolioComponents system-wide
KPVW	View Engine — filter, sort, group, pivot, board mode transforms
KPCM	Computation Engine — derived columns, analytical models, rollup aggregations
KPCL	Collaborative Editing — CRDT-backed multi-user real-time editing
KPEX	Export & Integration — CSV, XLSX, JSON, webhook, embed export
KLNK	Link Network — the inter-portfolio graph system: forests, trees, cross-user connections
KPID	Portfolio Identity System — multi-account, multi-identity, multi-profile management
PortfolioComponent	The universal node. Every row in any sheet is a PortfolioComponent (Item or Container).
PortfolioRow	A PortfolioComponent projected as a spreadsheet row: flat key-value map of ColumnId → TypedCellValue
SheetDefinition	A named, scoped, filterable view definition: base row type filter + visible column schema + default sort/group
ViewDefinition	A user-saved customization of a SheetDefinition: filters, sorts, groups, column widths, highlight rules, board config
CellValue	The typed value at a row-column intersection. Variants: Text Number Currency Date Enum Relation User Tag Bool Json Null
EventLog	Append-only, immutable log of every mutation on a PortfolioComponent. Never deleted.

Term	Definition
CrdtLog	LWW + OR-Set CRDT operation log. Applied on merge to resolve concurrent edits without data loss.
VectorClock	Logical clock per federation node. Provides causal ordering of all mutations.
InterPortfolioLink	A GraphEdge crossing the boundary between two different users' root spreadsheets. The atomic unit of KLNK.
LinkEdge	A typed, directional edge between two LinkNodes in the KLNK graph.
LinkForest	The complete set of all LinkTrees rooted at a given Sovereign Entity — the full picture of their network.
LinkTree	A rooted, directed subgraph of KLNK from a single root node to a configured depth.
ShadowRow	A read-only reflection in one user's spreadsheet of a linked component from another user's spreadsheet.
MirrorColumn	A subset of columns from a ShadowRow made visible as a ComputedColumn in the host spreadsheet.
Sovereign Entity	The real-world person, organization, collective, or cooperative that owns a root PortfolioSystem instance.
ProfilePartition	A logical partition of the root spreadsheet's rows, owned by one Identity. Rows are tagged with identity_tags.
VisibilityMask	A per-account or per-profile filter specifying which rows and columns are visible to which observer.
SplitPolicy	A governance policy governing how data is partitioned across identity partitions.
CrossIdentityMerge	The operation of unifying two identity partitions under one Sovereign Entity.
Oba	The Kogi platform AI assistant — the spreadsheet's AI chief-of-staff
kogi-engine	The Scala analytics and intelligence engine; consumes all portfolio events via gRPC/Kafka; writes back computed columns via WritebackService
WritebackService	The gRPC service that accepts engine-computed values and writes them to AIColumns, respecting all permission and policy checks
CRDT	Conflict-free Replicated Data Type — LWW + OR-Set — enables safe concurrent edits across all nodes and identities without conflicts